

Обработка естественного языка и машинный анализ текстов

Занятие 1. Attention. Transformer

Евгений Дарханов

Материалы

- Современный NLP: большие языковые модели: ссылка и ссылка
- Лекции одного дата-шрушера: ссылка и ссылка
- LLM-инженер: ссылка
- Информационный поиск и ранжирование: ссылка
- Lena Voita: ссылка

Содержание курса

1. Attention. Transformer
2. Large Language Models
3. Retrieval-Augmented Generation
4. Agents

Содержание занятия

1. Recap
2. Attention
3. Transformer

01

Recap

Токенизация

- **Токен** – минимальная структурная единица текста (последовательность символов)
- **Словарь** – непустое конечное множество токенов
- **Текст** – это последовательность токенов
- **Токенизация** – процесс представления текста в виде последовательности **токенов**

Токенизация

- **Посимвольно:** маленький словарь, очень длинные последовательности

V* = {_, *а, б, в, г, д, е, ё, ж, з, и, й, к, л, м, н, о, п, р, с, т, у, ф, к, ц, ч, ш, щ, ъ, ы, ь, э, ю, я}

Текст	Последовательность токенов	Последовательность номеров
мама мыла раму	м,а,м,а,_,м,ы,л,а,_,р,а,м,у	14,1,14,1,0,14,29,13,1,0,18,1, 14,21
бобры добры добротой бобры полны	б,о,б,р,ы,_,д,о,б,р,ы,_,д,о,б,р,о,т ,о,й,_,б,о,б,р,ы,_,п,о,л,н,ы	3,16,3,18,30,0,6,16,3,18,30, 6,16,3,18,16,20,16,12, 3,16,3,18,30,0,17,16,13,15,30

Токенизация

- **Пословно:** огромный словарь, специальные токены для новых или редких слов

***V* = {_, *мама, мыла, раму, бобры, добры, добротой, полны*}**

Текст	Последовательность токенов	Последовательность номеров
мама мыла раму	мама,_,мыла,_,раму	1,0,2,0,3
бобры добры добротой бобры полны	бобры,_,добры,_,добротой,_,бобры,_,полны	4,0,5,0,6,0,4,0,7

Токенизация

- **По частям слов:** промежуточный вариант

$$V = \{_, ма, мы, ла, ра, му, боб, ры, доб, рот, ой, пол, ны\}$$

Текст	Последовательность токенов	Последовательность номеров
мама мыла раму	ма,ма,_,мы,ла,_,ра,му	1,1,0,2,3,0,4,5
бобры добры добротой бобры полны	боб,ры,_,доб,ры,_,доб,рот,ой,_, боб,ры,_,пол,ны	6,7,0,8,7,0,8,9,10,0,6,7,0,11,12

Byte-Pair Encoding

aabaaaba → a**ab**aa**ab**a → **[ab = X]** → a**X**aa**X**a → **[aX = Y]** → **Y**a**Y**a → **[Ya = Z]** → **ZZ**

1. Добавляем в словарь все уникальные символы и специальные токены
2. Пока размер словаря не достигнет заданного размера повторяем:
   - Находим наиболее частотную пару символов во всех текстах
   - Объединяем найденную пару в новый токен
   - Заменяем во всех текстах все вхождения пары на новый токен
3. Для токенизации нового слова последовательно выполняем замены символов в порядке выполнения

https://arxiv.org/pdf/2411.08671

Byte-Pair Encoding на уровне байтов

- Пусть исходный словарь – это все байты от 0 до 255
- Закодируем исходные тексты в UTF-8, получим набор байтов и соответствующие токены
- Далее запускаем алгоритм BPE

Пример: слово «**кот**»

- буква «**к**» → `(0xD0, 0xBA)`
- буква «**о**» → `(0xD0, 0xBE)`
- буква «**т**» → `(0xD1, 0x82)`

Byte-Pair Encoding на уровне байтов

- Исходная цепочка байтов для «**кот**»: *[0xD0, 0xBA, 0xD0, 0xBE, 0xD1, 0x82]*

- Слово «**кот**» – это последовательность из 6 отдельных токенов:
  [T(0xD0), T(0xBA), T(0xD0), T(0xBE), T(0xD1), T(0x82)]

- Слияние (буква «**о**»): *(0xD0, 0xBE) = T_new_1*
- Последовательность: *[T(0xD0), T(0xBA), T_new_1, T(0xD1), T(0x82)]*

Специальные токены

- **<bos>** – токен начала последовательности
- **<eos>** – токен конца последовательности
- **<pad>** – токен для выравнивания последовательностей по длине

Пример:

- [<bos>, бобры, _, добры, _, добротой, _, бобры, _, полны, <eos>]
- [<bos>, мама, _, мыла, _, раму, <eos>, <pad>, <pad>, <pad>, <pad>]

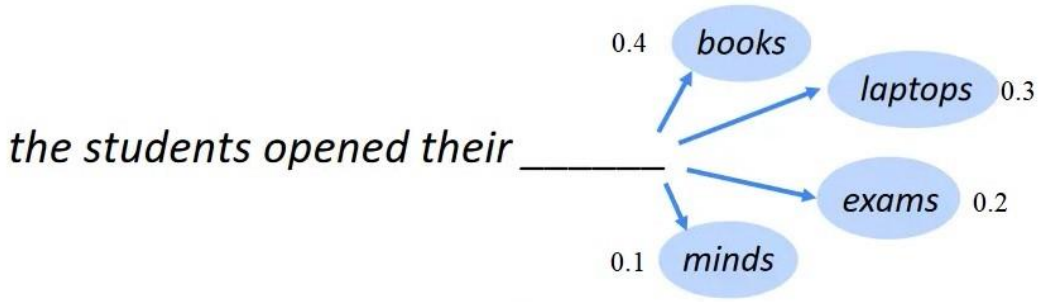
Задача языкового моделирования

Языковая модель (*Language Model, LM*) – это вероятностная модель языка

Имея языковую модель, можно:

- оценивать вероятность текста (последовательности)
- предсказывать следующее слово (токен)

$$p(x_t | x_0, x_1, \ldots, x_{t-1})$$



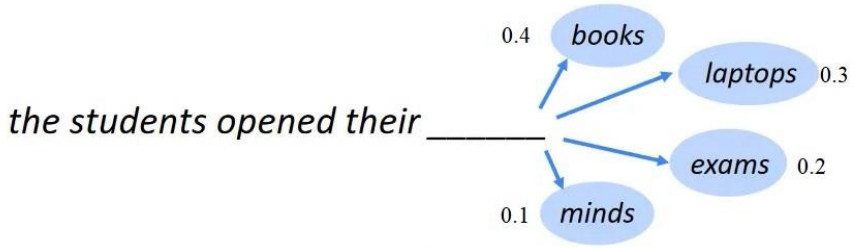
https://medium.com/@antonio.lopardo/the-basics-of-language-modeling-1c8832f21083

Генерация последовательности

Жадная генерация (*Greedy Decoding*)

- Модель всегда выбирает наиболее вероятный токен

$$x_t = \underset{x \in V}{\operatorname{argmax}} \mathcal{P}(x | x_0, x_1, \ldots, x_{t-1})$$



Случайная генерация (*Sampling*)

- Продолжаем последовательность, семплируя токен из условного распределения

$$x_t \sim \mathcal{P}(x | x_0, x_1, \ldots, x_{t-1})$$

Генерация последовательности. Случайная

top-k:

- семплируем только из ***k*** наиболее вероятных токенов

top-p:

- семплируем из минимального набора (ядро) наиболее вероятных токенов, сумма вероятностей которых больше или равна заданному порогу ***p***

Семплирование с температурой

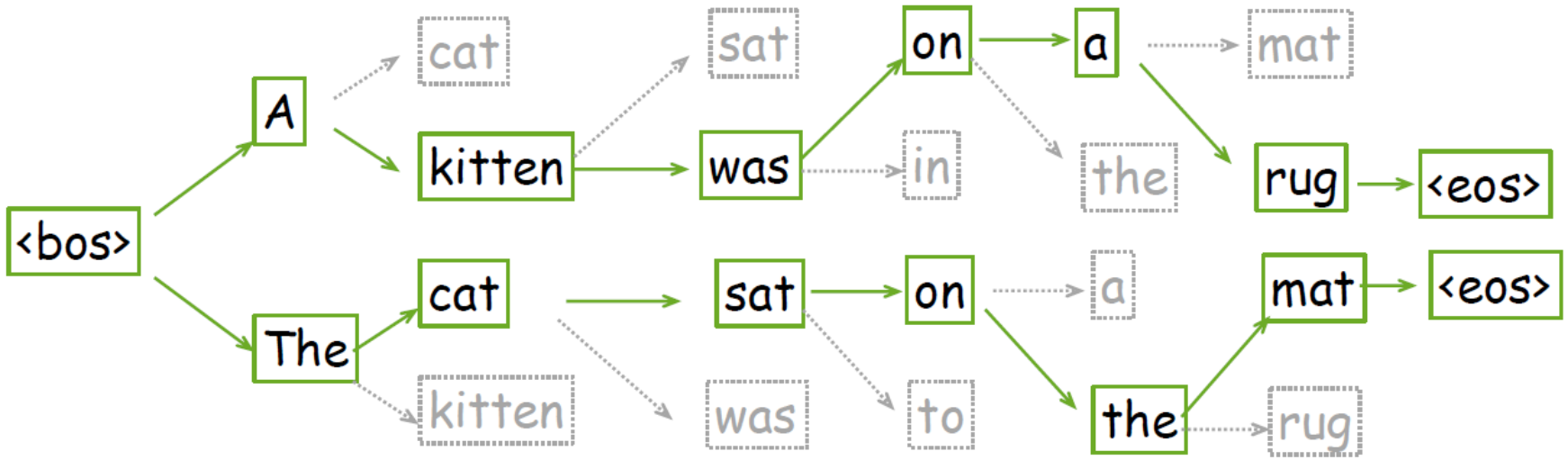
$$p_i = \text{softmax}(z_i) = \frac{e^{\frac{z_i}{T}}}{\sum_{j=1}^{V} e^{\frac{z_i}{T}}}$$

- ***T = 1***. Классический *softmax*
- ***T → 0***. Модель превращается в жадный декодинг
- ***T > 1***. Вектор вероятностей стремится к равномерному распределению

Генерация последовательности. Жадная

Beam Search

- Поддерживаем несколько самых вероятных траекторий
- Количество траекторий является гиперпараметром



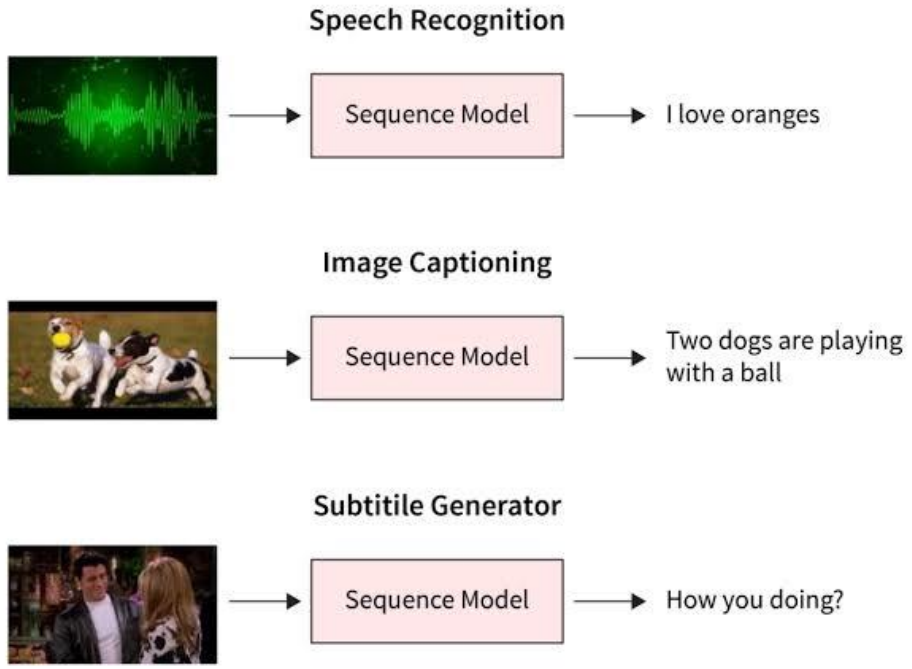
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Задача sequence to sequence

Для входной последовательности необходимо сгенерировать выходную последовательность. Длины последовательностей могут различаться

Примеры:

- Машинный перевод
- Суммаризация текста
- Генерация ответа на вопрос



https://dilipkumar.medium.com/sequence-to-sequence-seq2seq-models-37318a342b1d

Задача sequence to sequence

Алгоритм:

- Токенизатор разбивает входной текст на токены
- Токены вместе с **<bos>** идут на вход в LM
- LM отдаёт логиты
- Выбранным алгоритмом семплируем токен из распределения
- Новый токен добавляется к контексту. Токены снова отправляются на вход в LM
- Получаем распределение на следующий токен
- Продолжаем до **<eos>**

Задача sequence to sequence

Алгоритм:

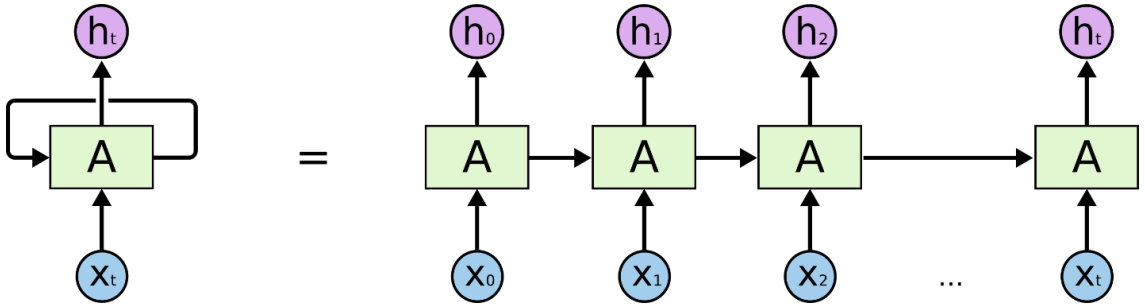
- Токенизатор разбивает входной текст на токены
- Токены вместе с **<bos>** идут на вход в LM
- LM отдаёт логиты
- Выбранным алгоритмом семплируем токен из распределения
- Новый токен добавляется к контексту. Токены снова отправляются на вход в LM
- Получаем распределение на следующий токен
- Продолжаем до **<eos>**

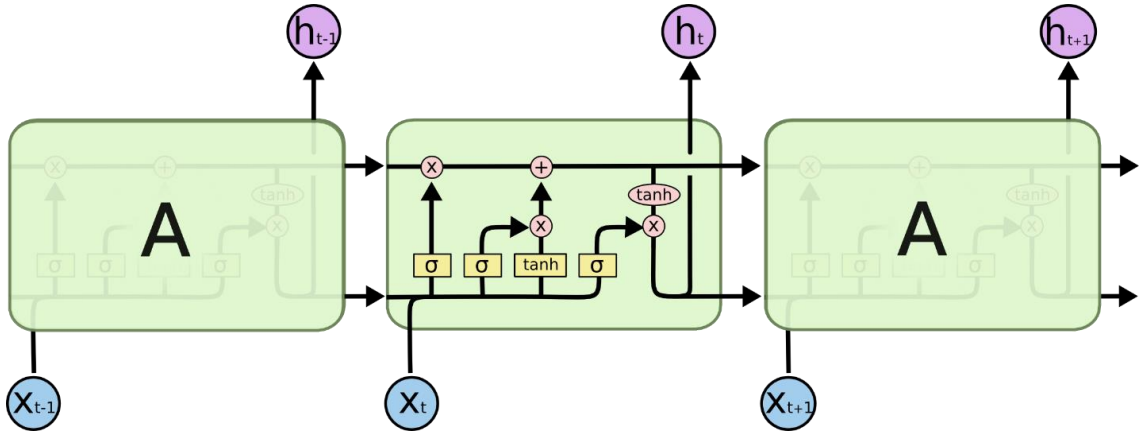
Что если в качестве языковой модели взять нейронную сеть?

Задача sequence to sequence

Обычные **RNN/LSTM/GRU** не подойдут

- Короткий контекст → теряем знания
- Вектор скрытого состояния регулярно перезаписывается → конфликт режимов работы

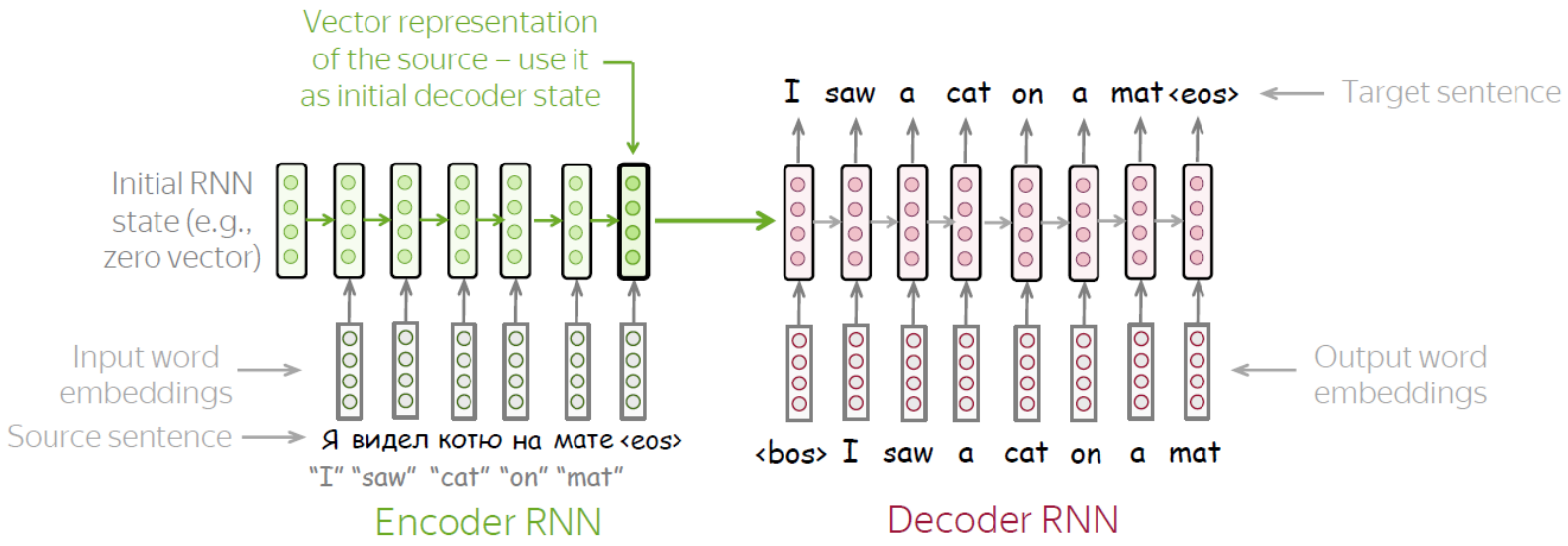




Encoder-Decoder

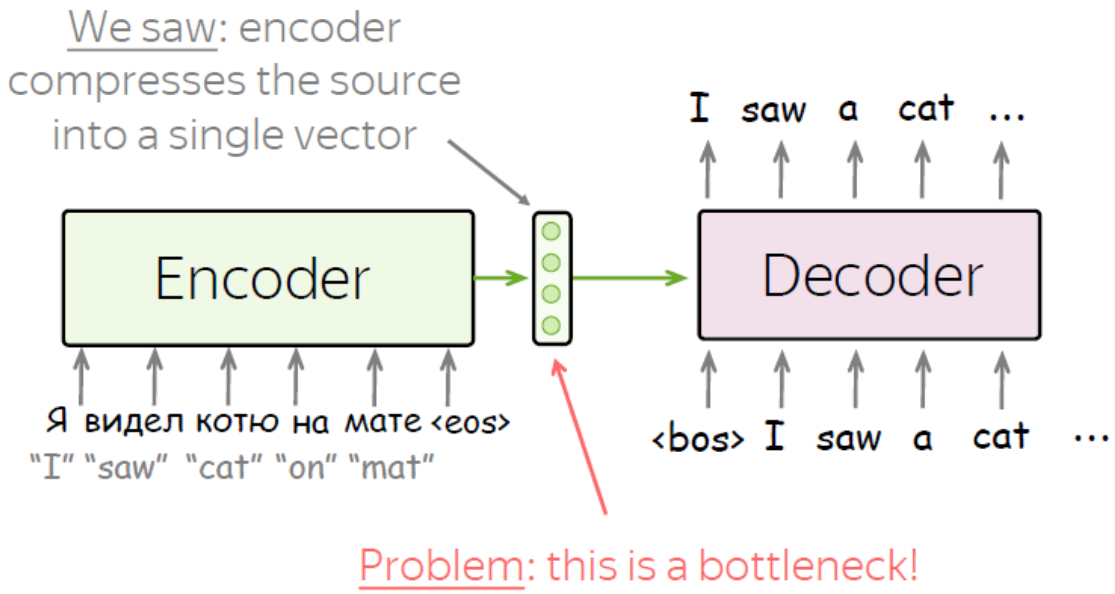
Модель состоит из двух частей: кодировищик (*encoder*) и декодировщик (*decoder*)

- Левая часть читает, пытается понять смысл и закодировать текст
- Правая часть генерирует на основе левой



https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Encoder-Decoder



- Теряется много информации при сжатии в вектор
- Декодеру может понадобиться информация о разных частях предложения

https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

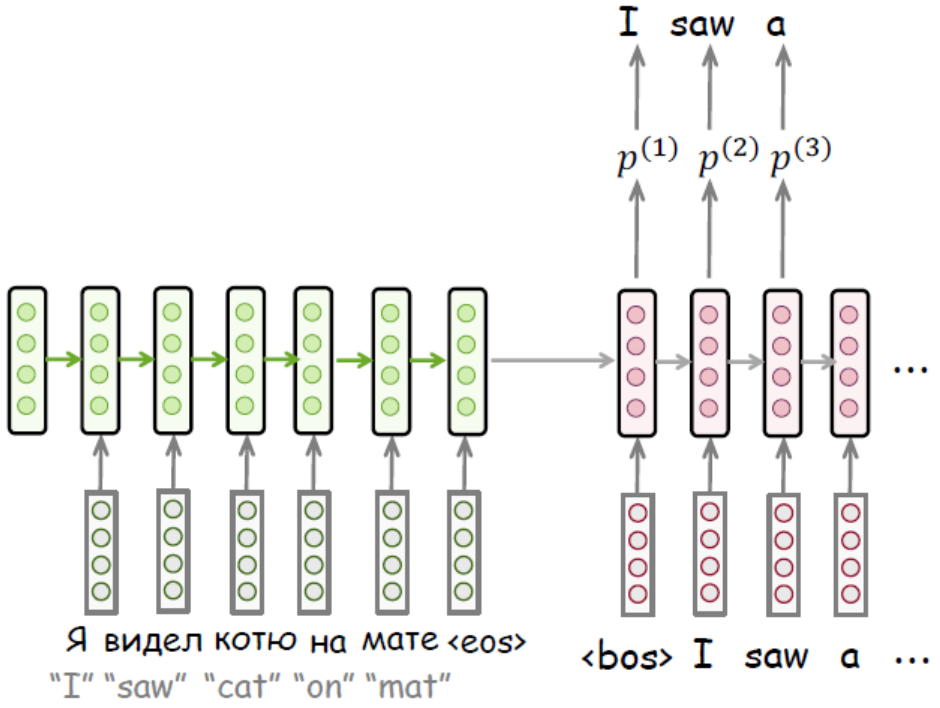
02

Attention

Attention

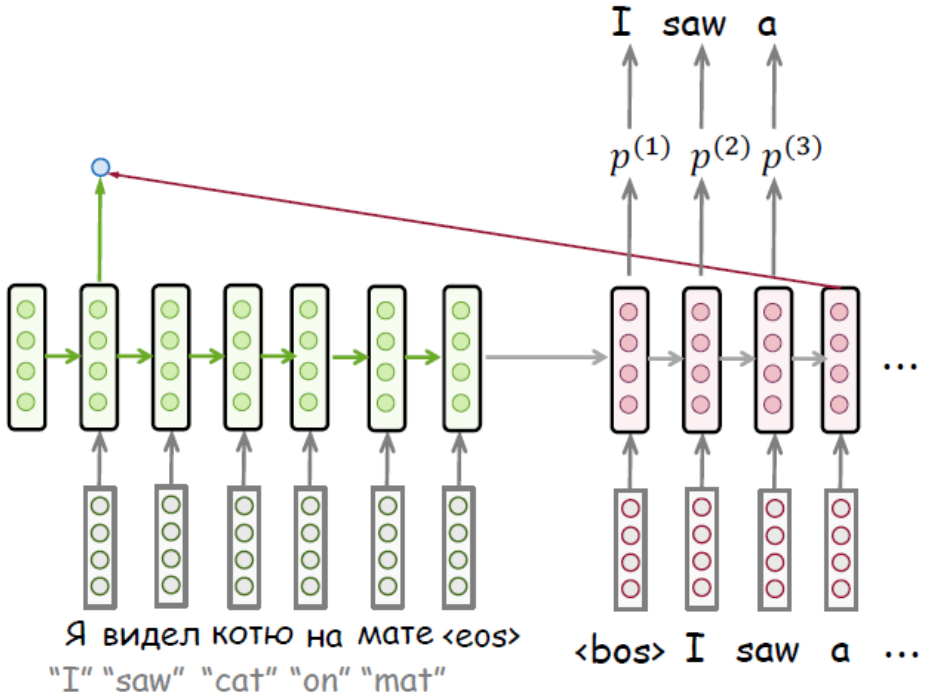
На разных шагах позволим модели сфокусироваться на разных входных токенах

- Будем запоминать все скрытые состояния h_{enc}
- На этапе декодирования будем брать конкретный h_{dec} и все h_{enc}, и смотреть на их взаимодействие



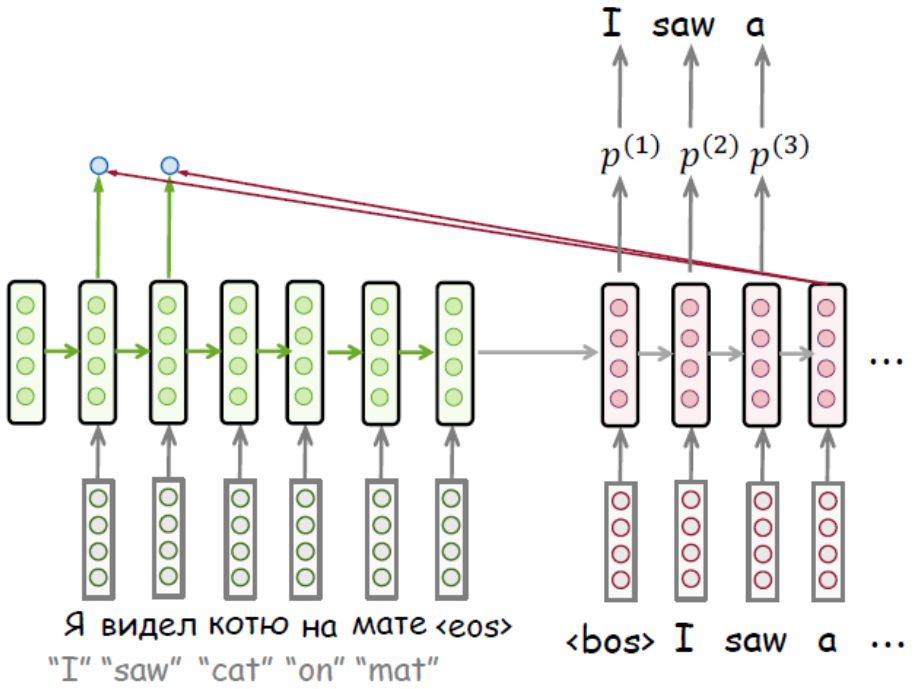
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention



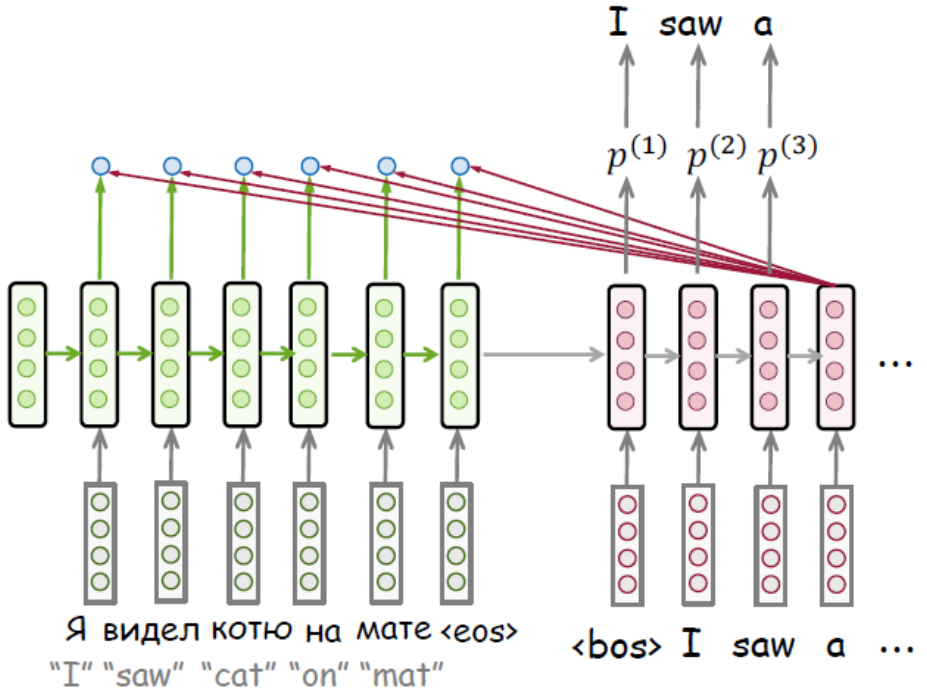
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention



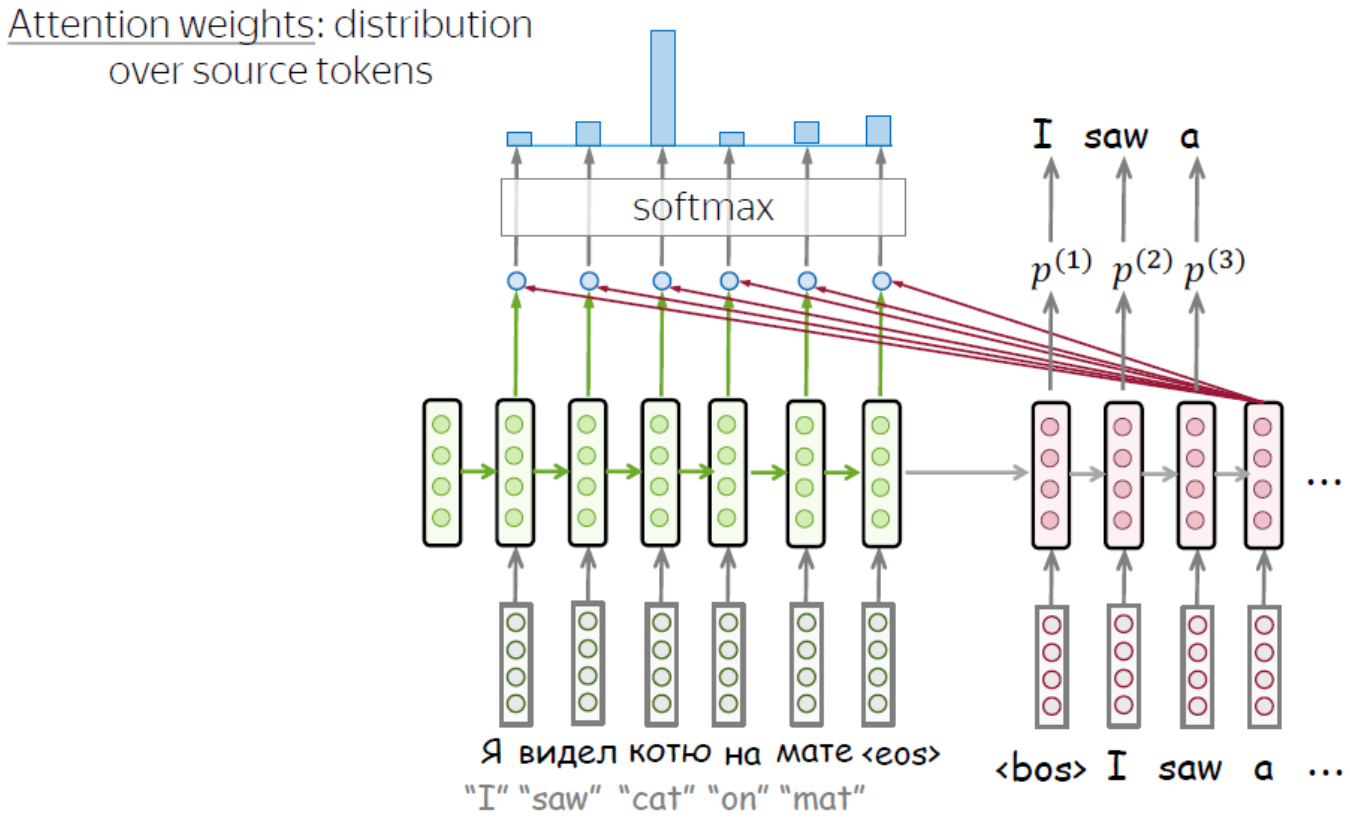
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention



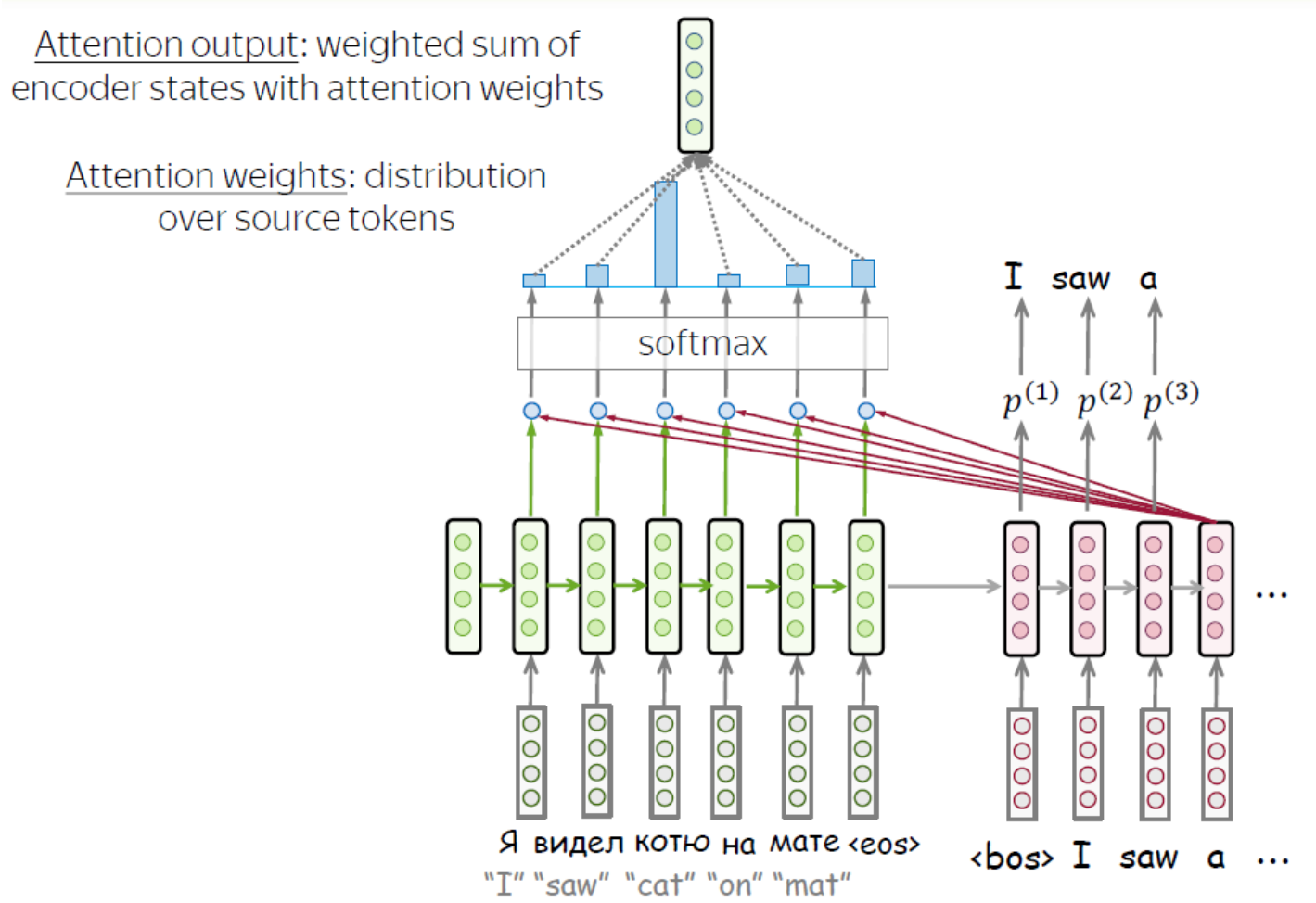
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention



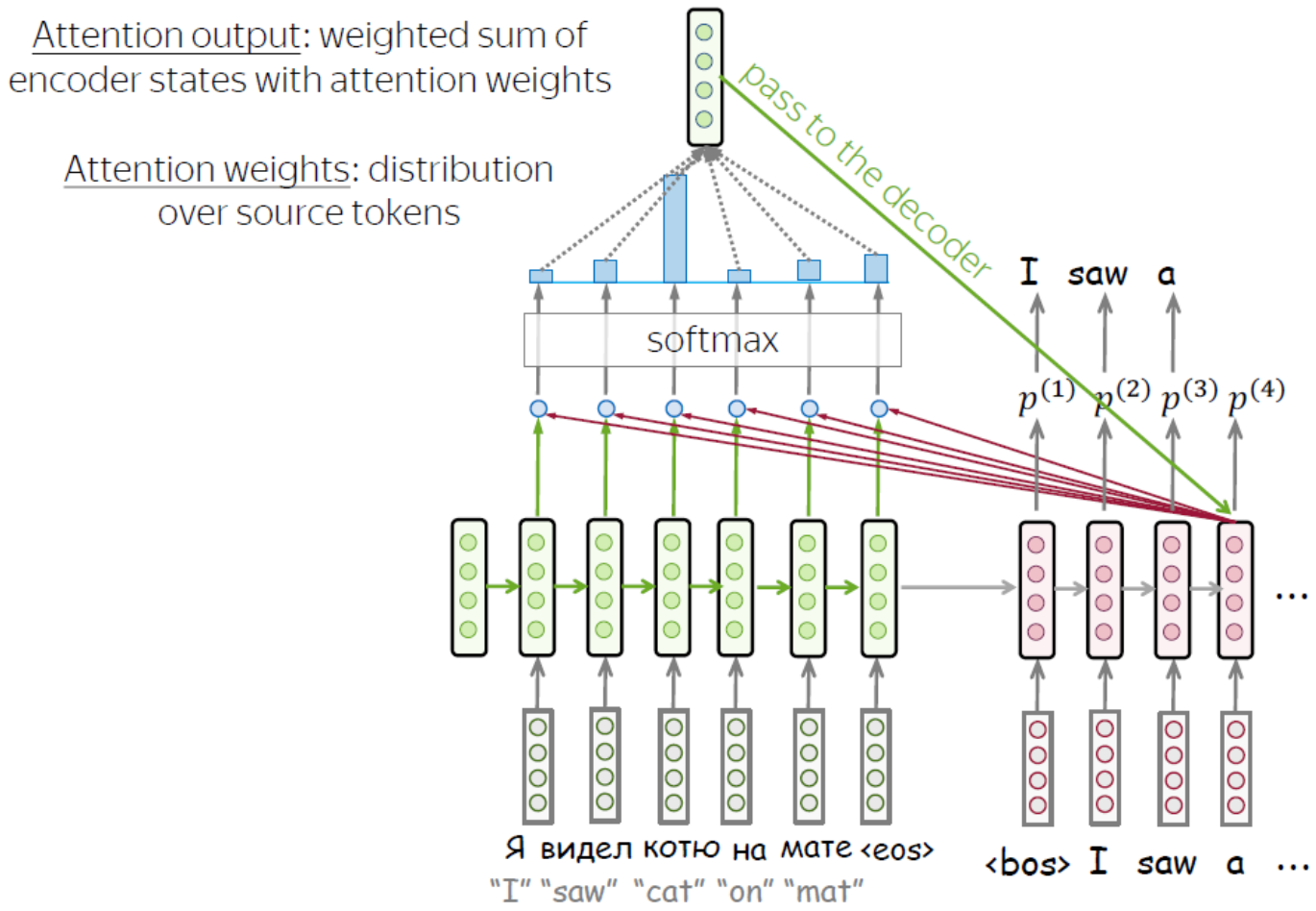
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention



https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention



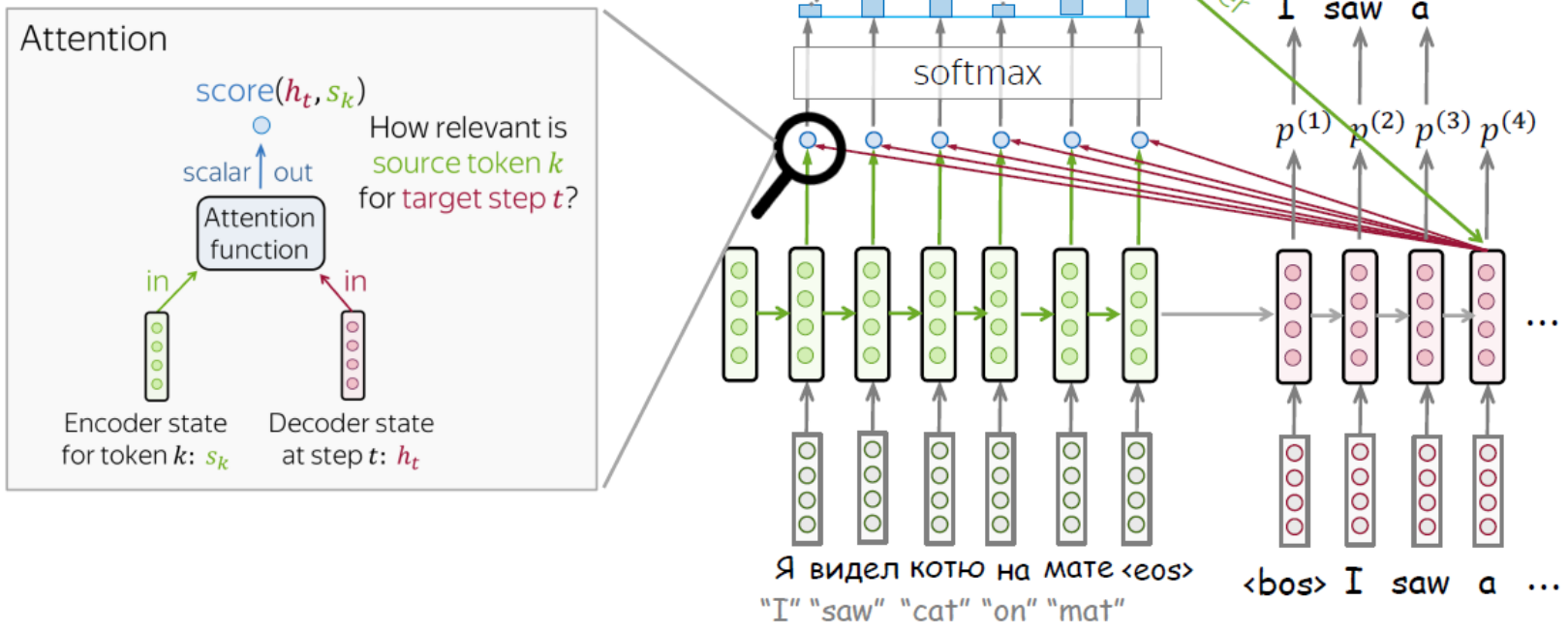
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention

два вектора $(s_k, h_t) \rightarrow [attention\ function] \rightarrow$

скаляр $\text{score}(s_k, h_t) \rightarrow [softmax] \rightarrow$

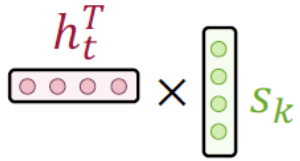
значимость s_k для h_t



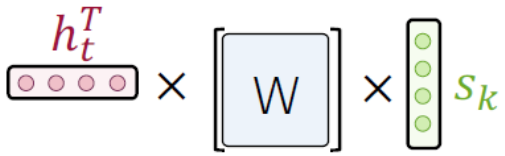
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention

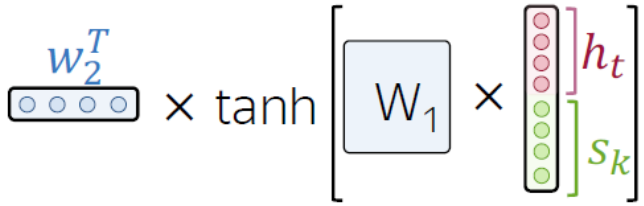
- Dot-product: $\text{score}(h_t, s_k) = h_t^T \, s_k$

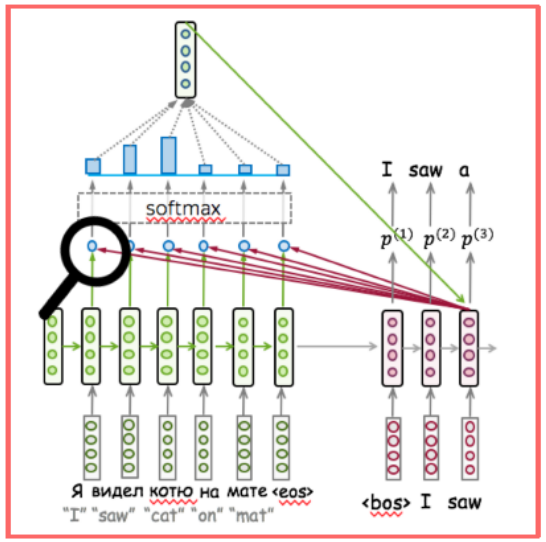


- Bilinear: $\text{score}(h_t, s_k) = h_t^T \, W s_k$



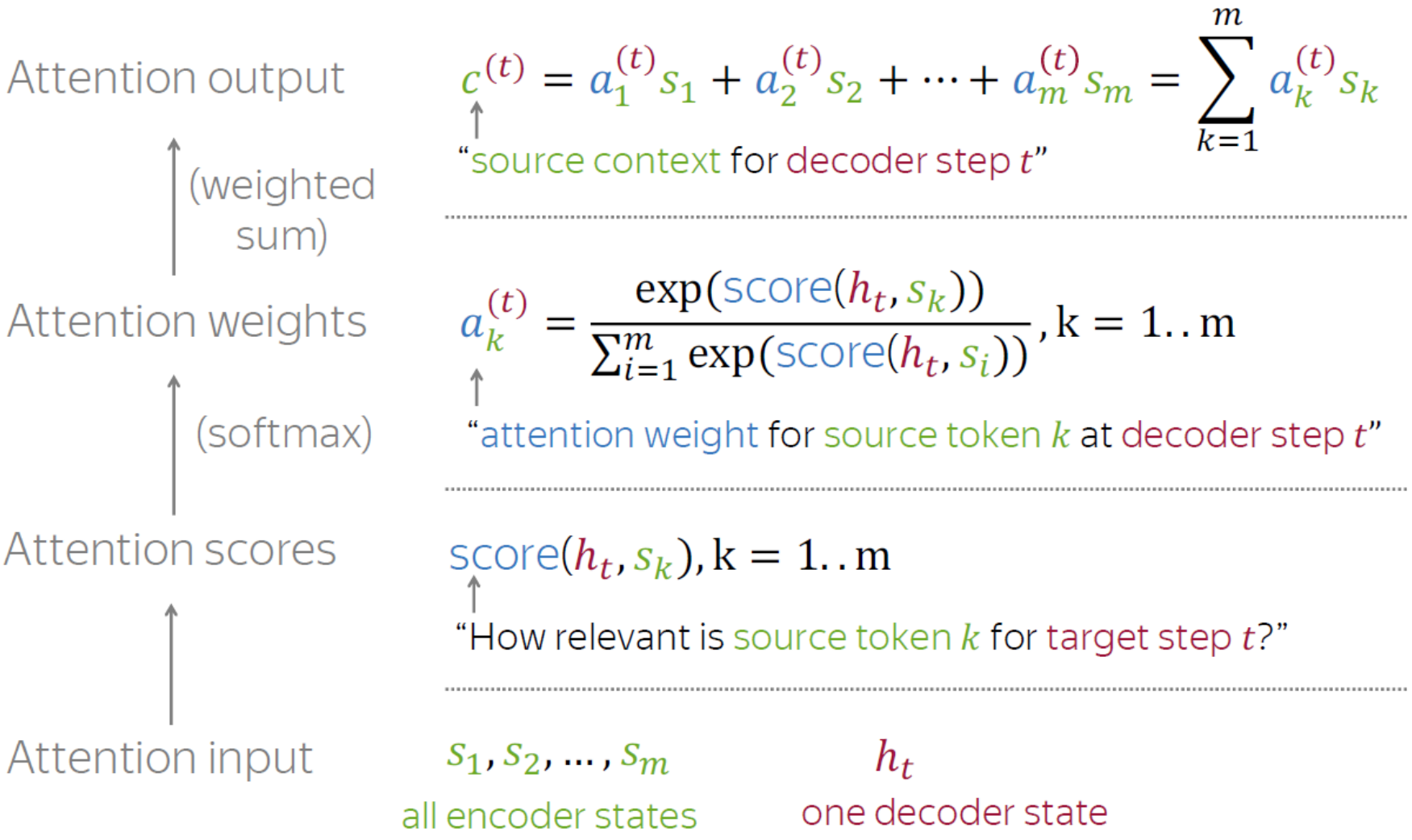
- Multi-Layer Perceptron: $\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1 [h_t, s_k])$





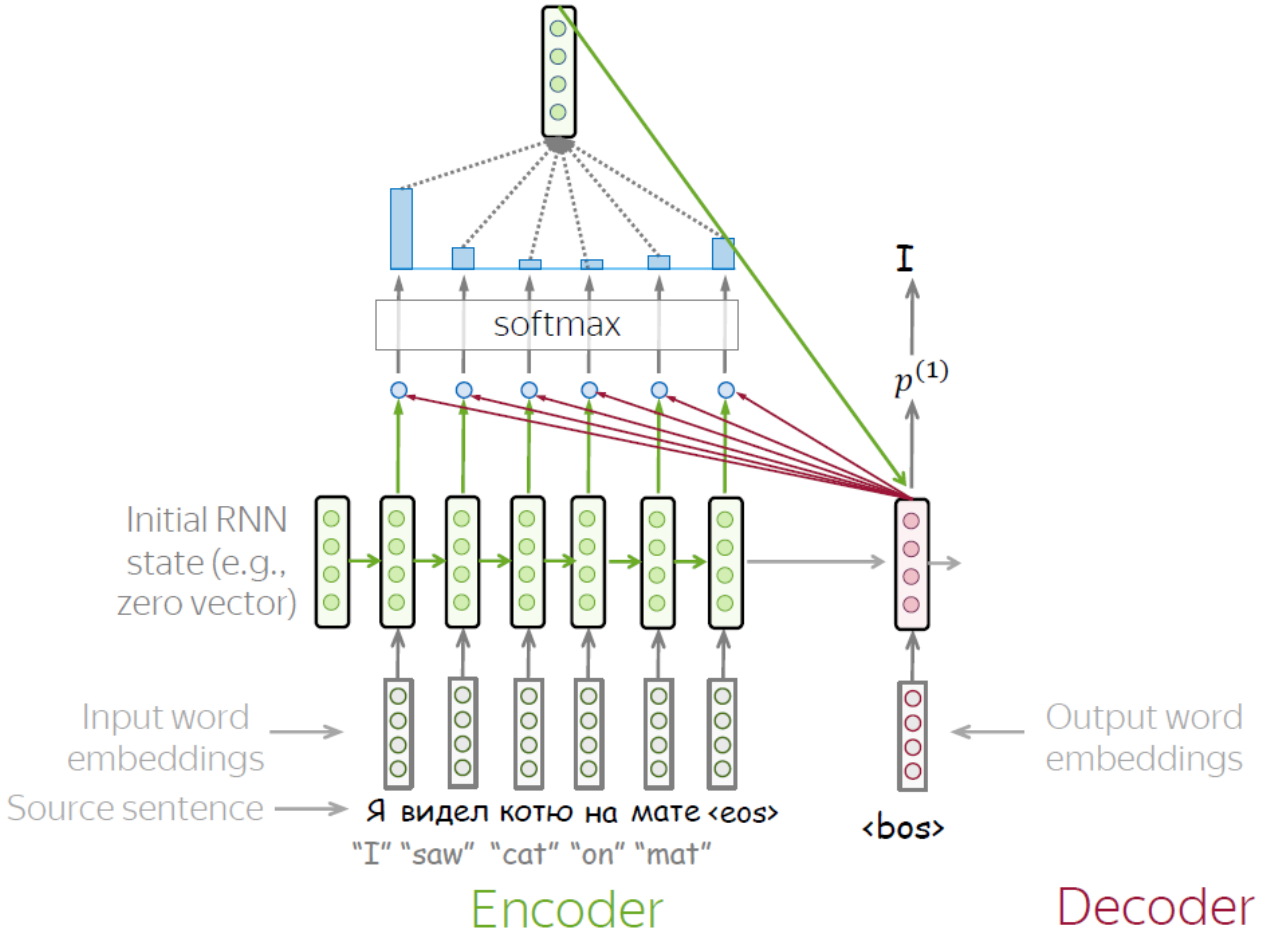
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention



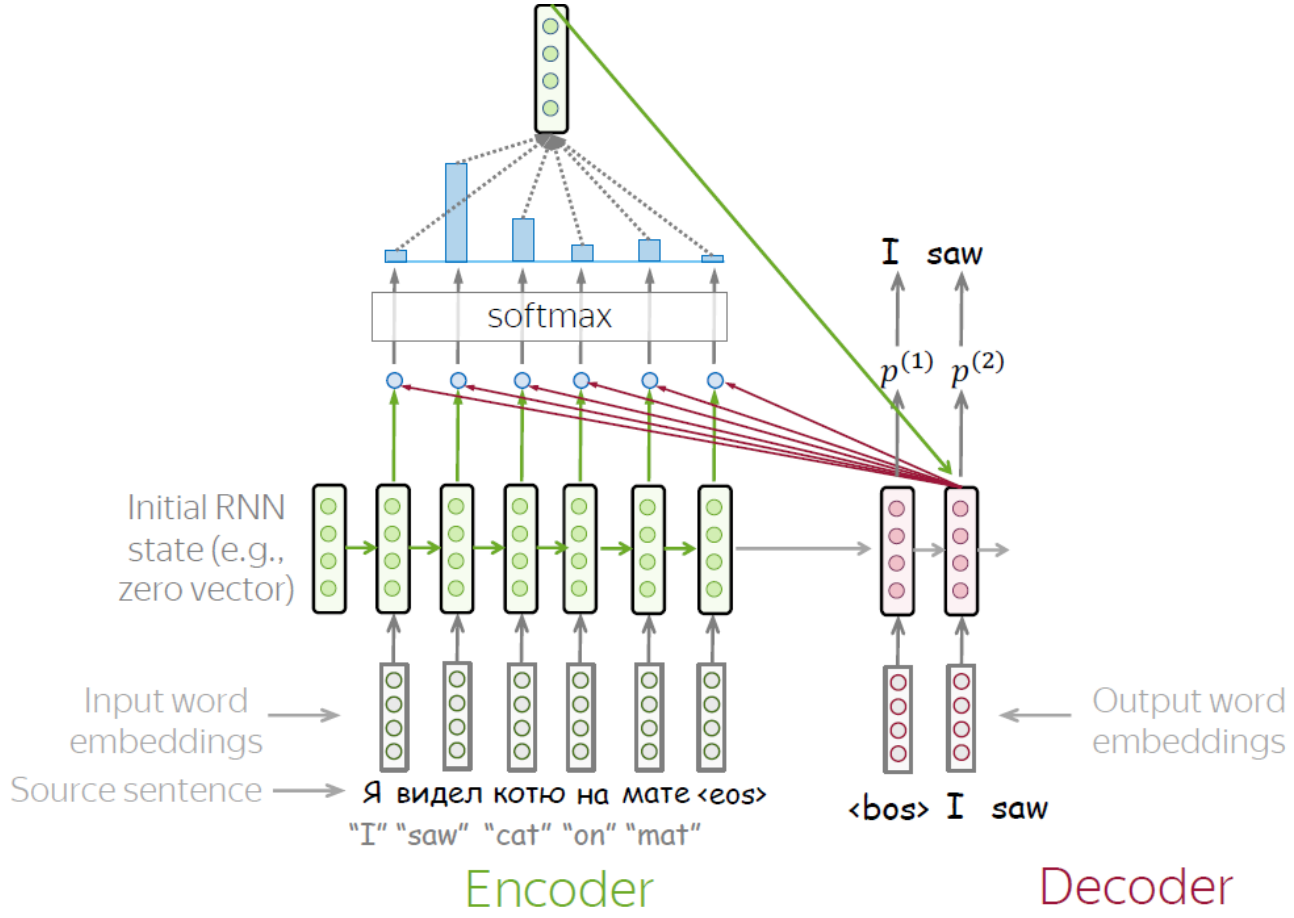
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention. Decoding



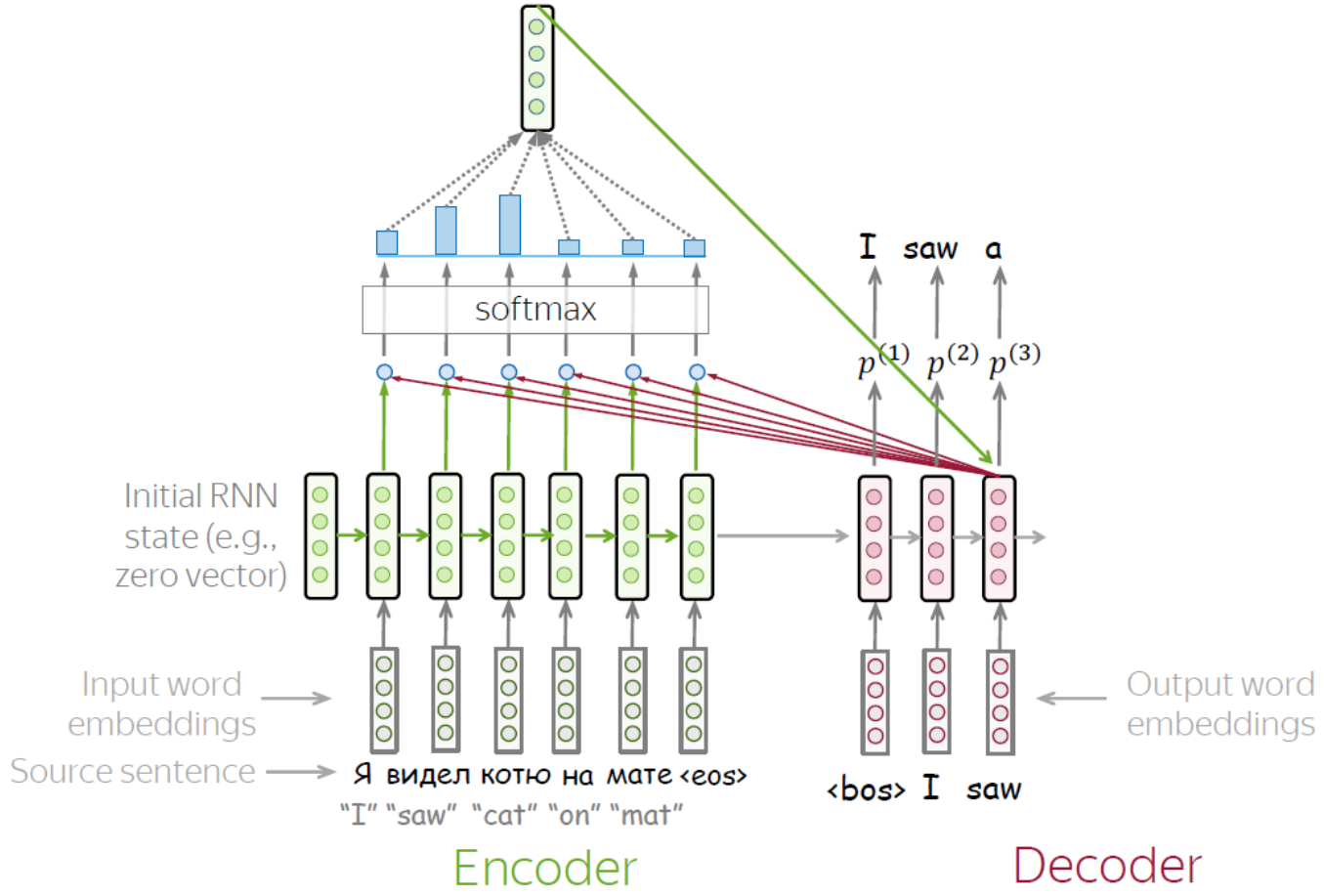
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention. Decoding



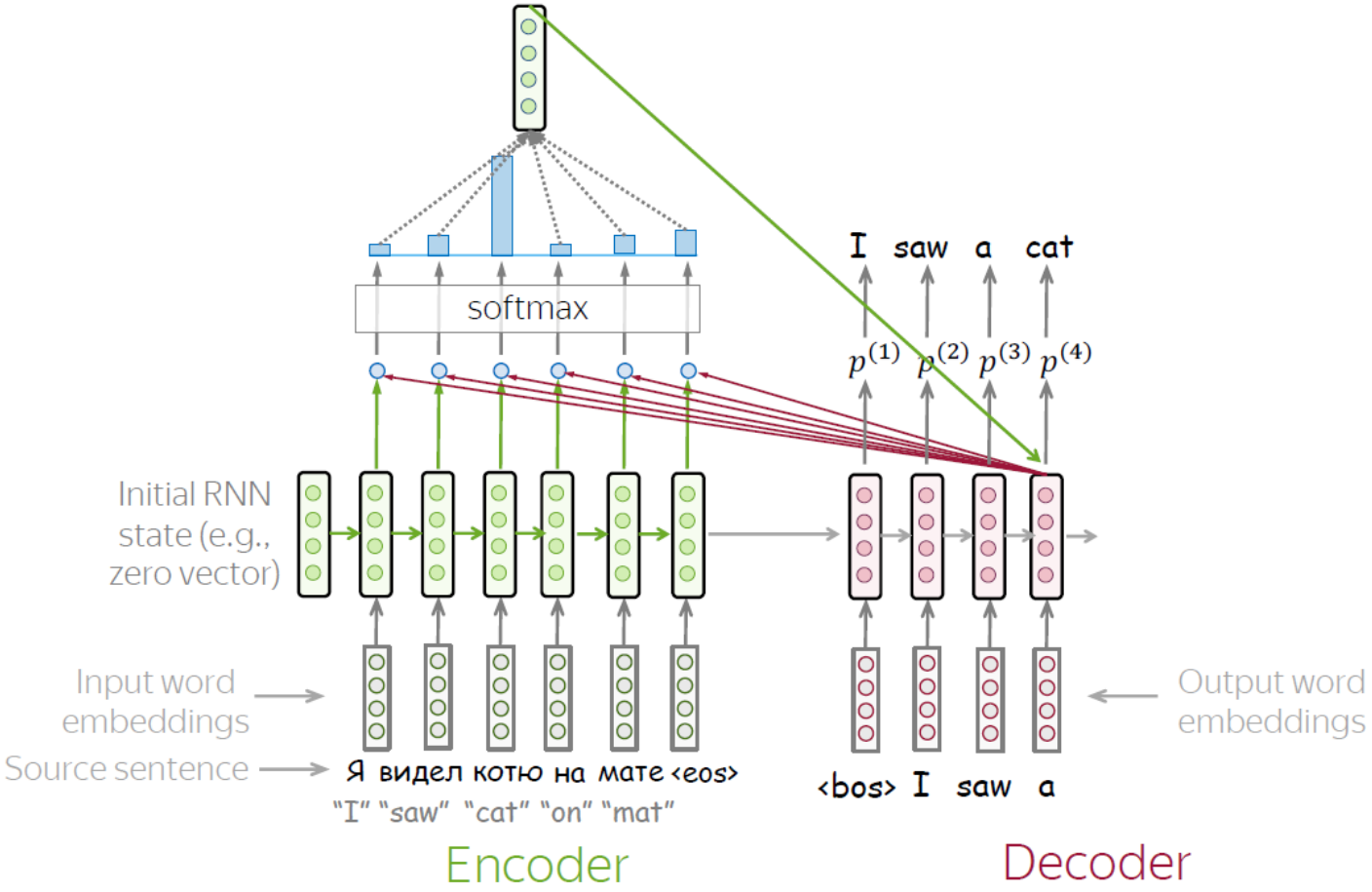
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention. Decoding



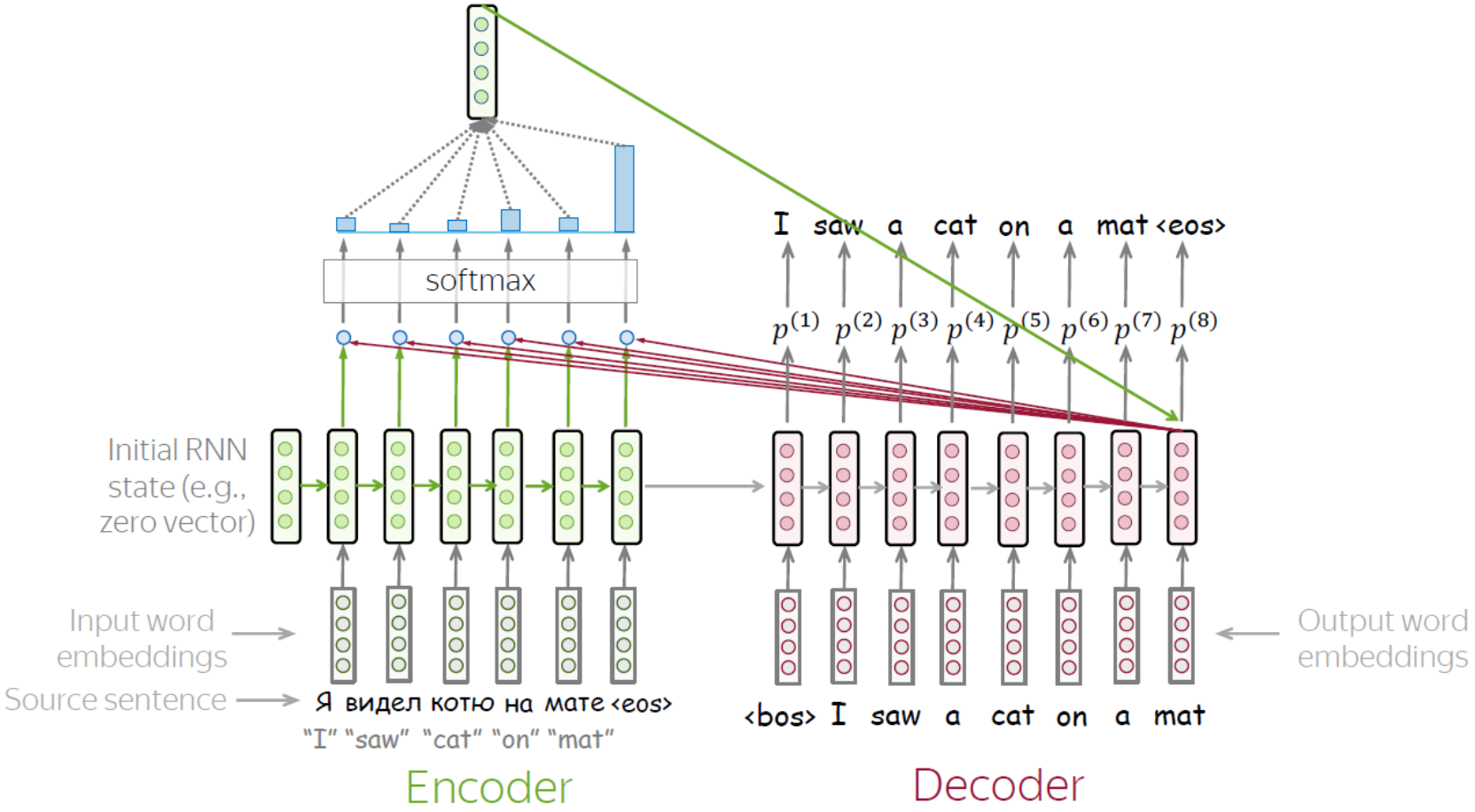
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention. Decoding



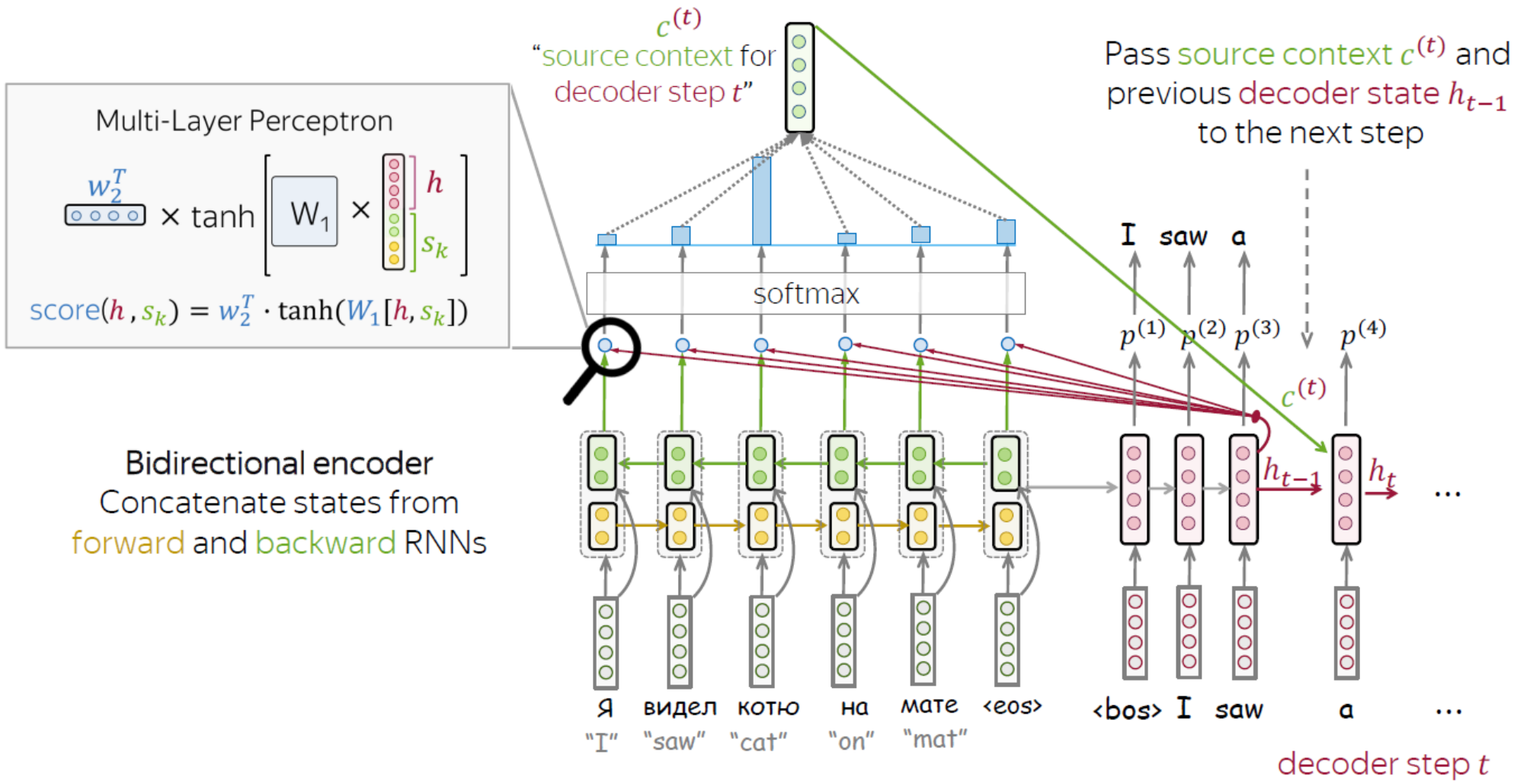
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Attention. Decoding



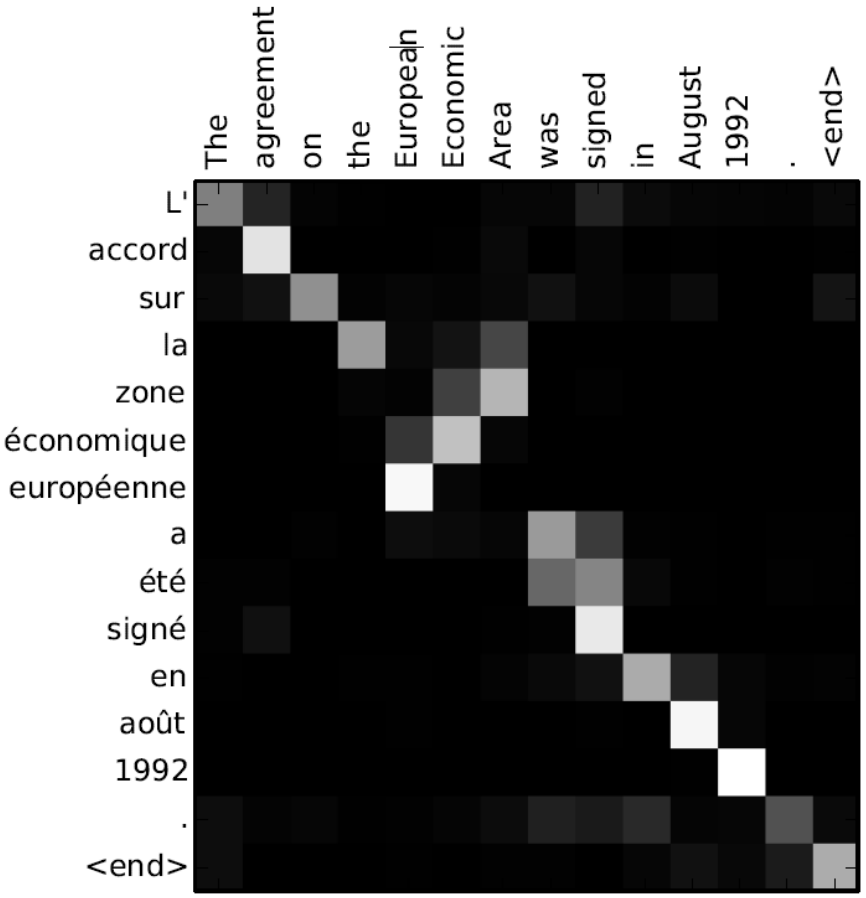
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

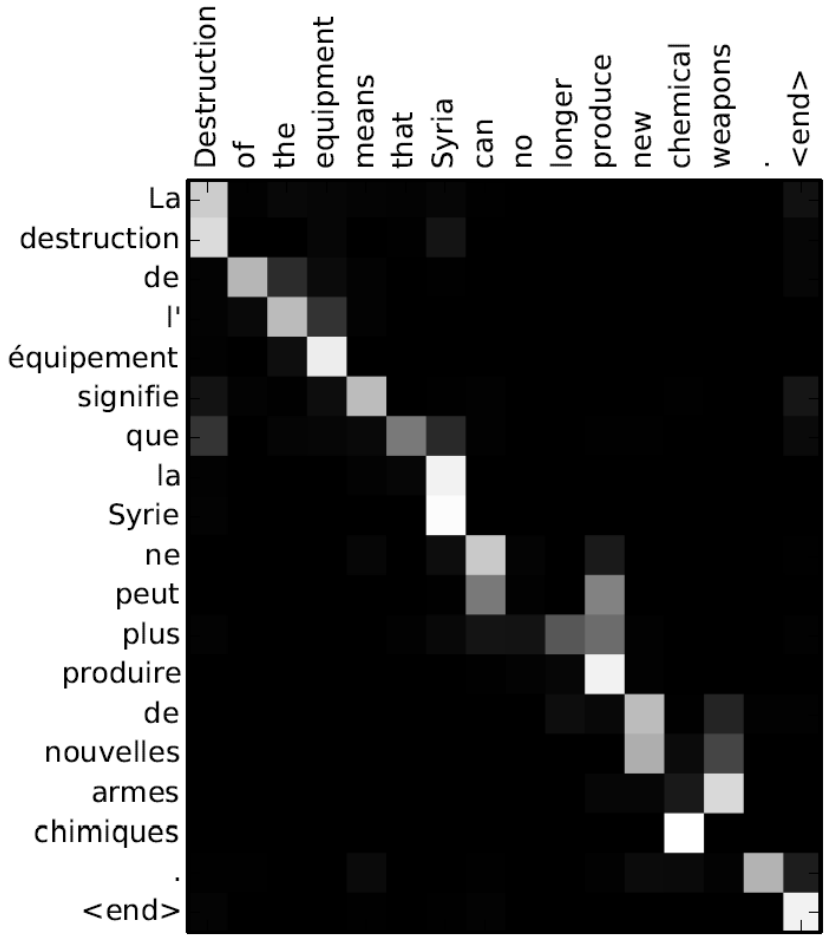
The Bahdanau Attention Mechanism, 2014



https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

The Bahdanau Attention Mechanism, 2014





https://arxiv.org/pdf/1409.0473

03

Transformer

Attention Is All You Need, 2017

Attention (2014)

- Невозможность параллелизации вычислений
- Ограниченная «память» скрытого состояния
- Однобокий контекст

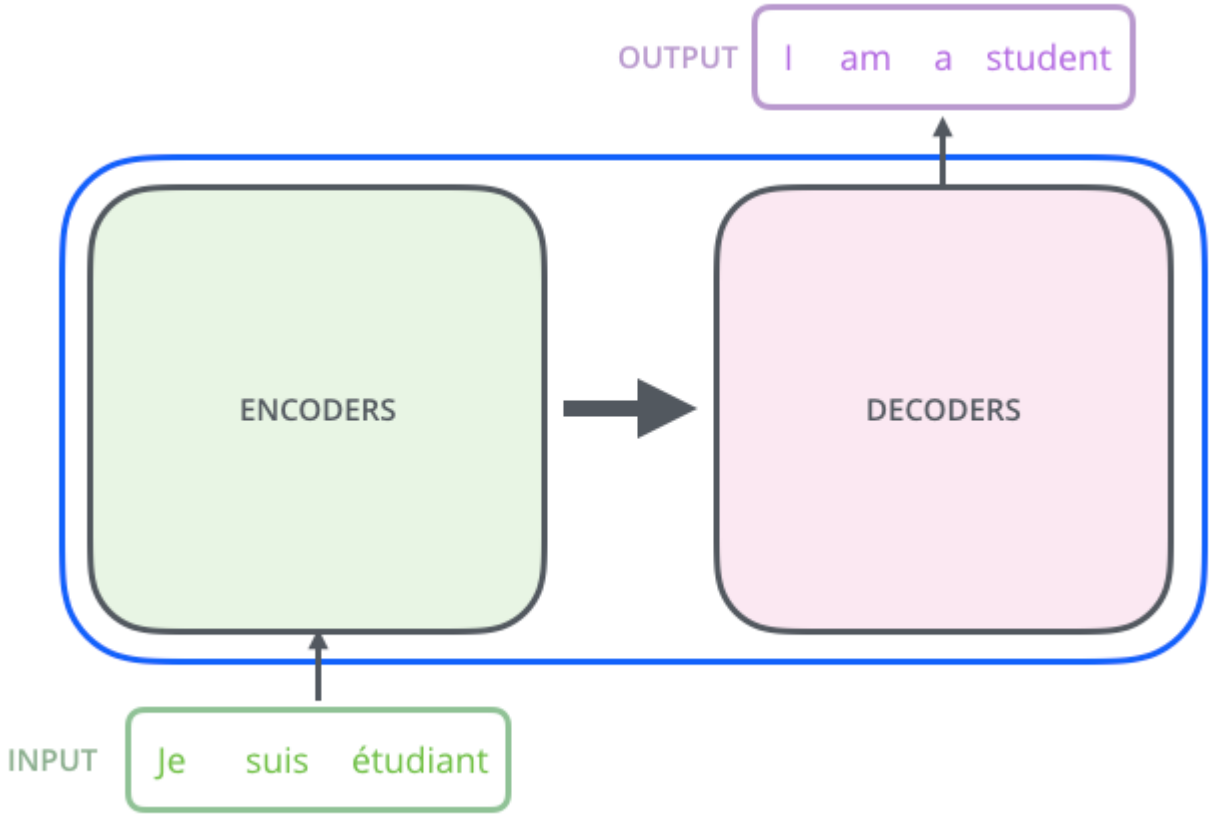
Transformer (2017)

- Новая архитектура для задач Seq2Seq, основанная только на полносвязных слоях
- Главный элемент – **Multi-Head Attention**

https://arxiv.org/pdf/1706.03762

Transformer

Энкодер и декодер

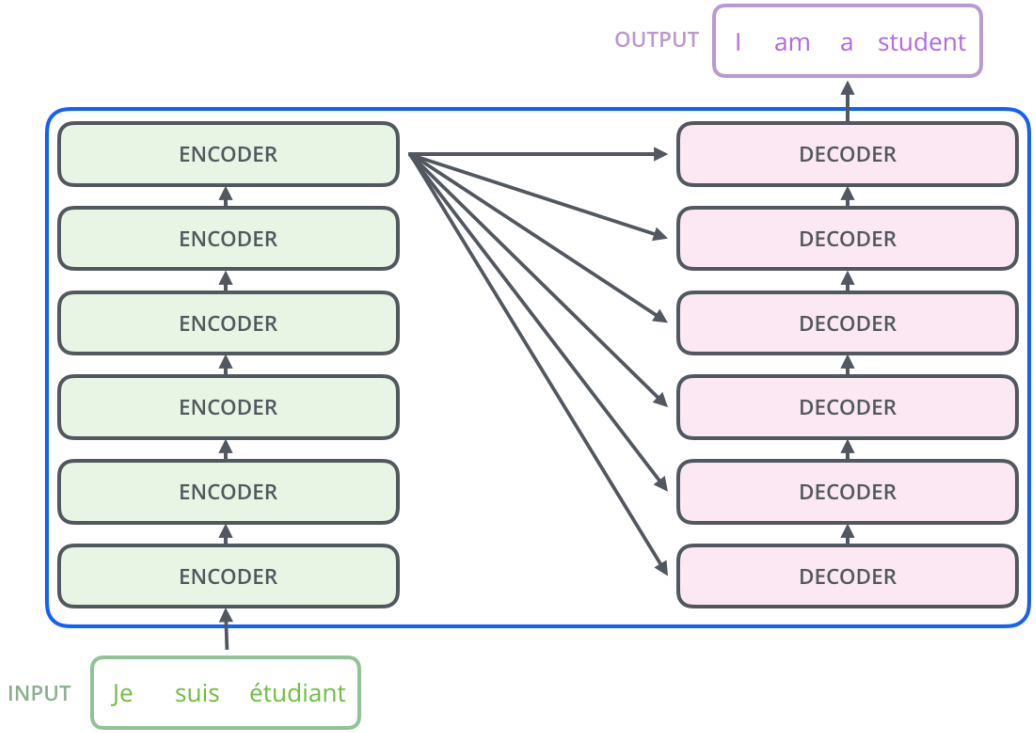


https://jalammar.github.io/illustrated-transformer/

Transformer

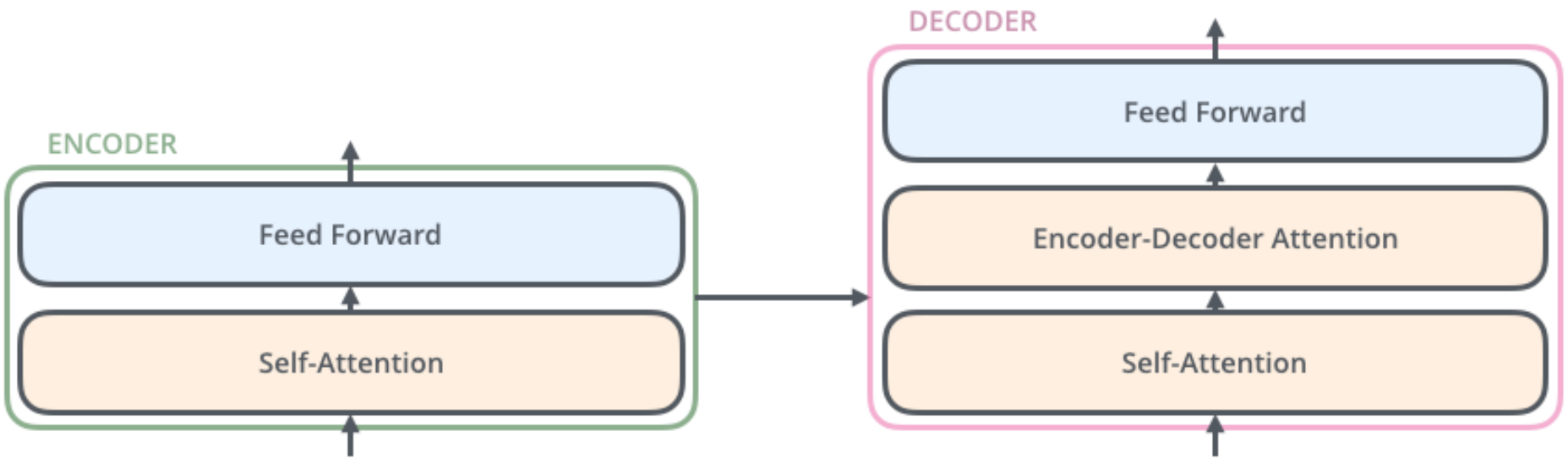
Энкодер и декодер состоят из одинаковых блоков. Веса в блоках разные.

После каждого слоя эмбеддинги становятся более информативными



https://jalammar.github.io/illustrated-transformer/

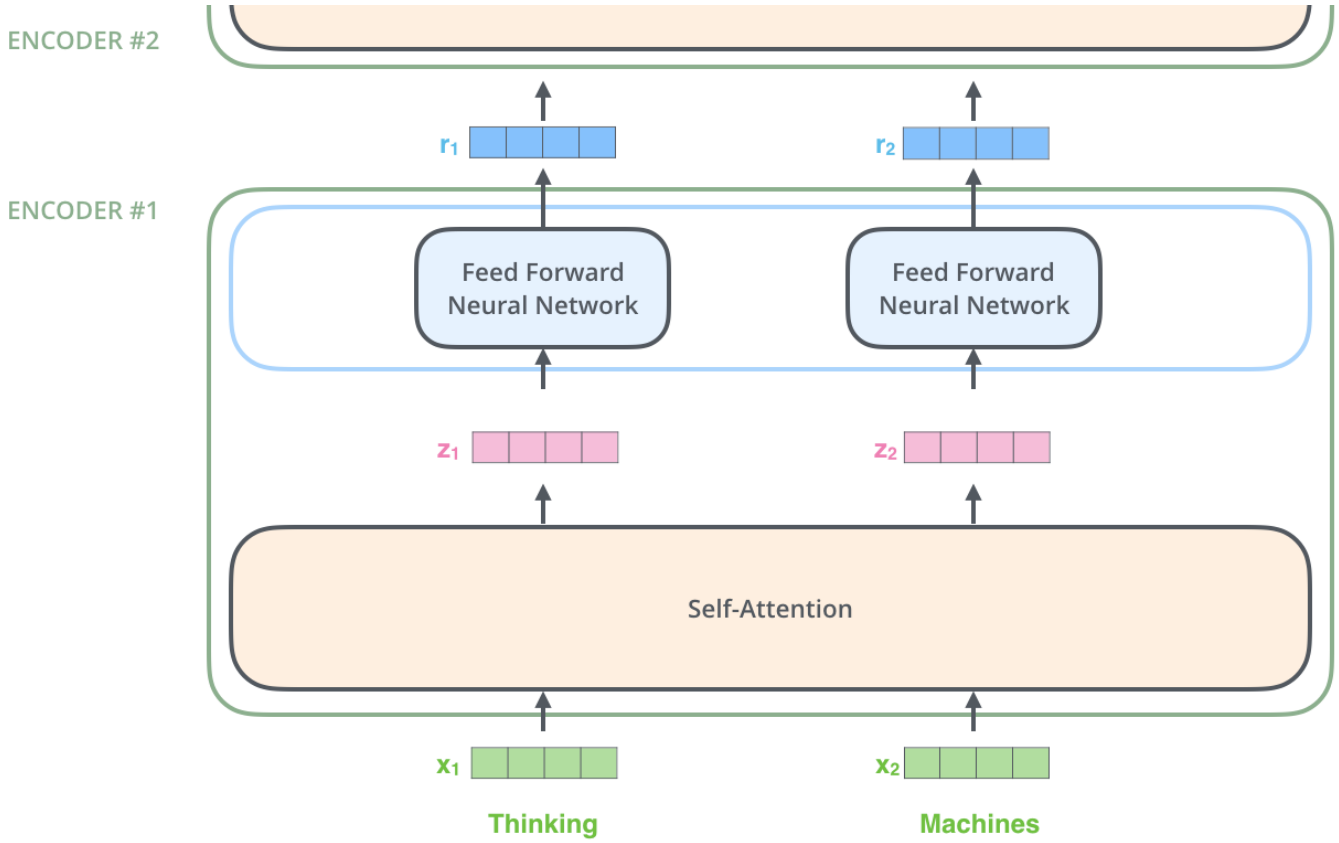
Transformer



https://jalammar.github.io/illustrated-transformer/

Transformer. Encoder

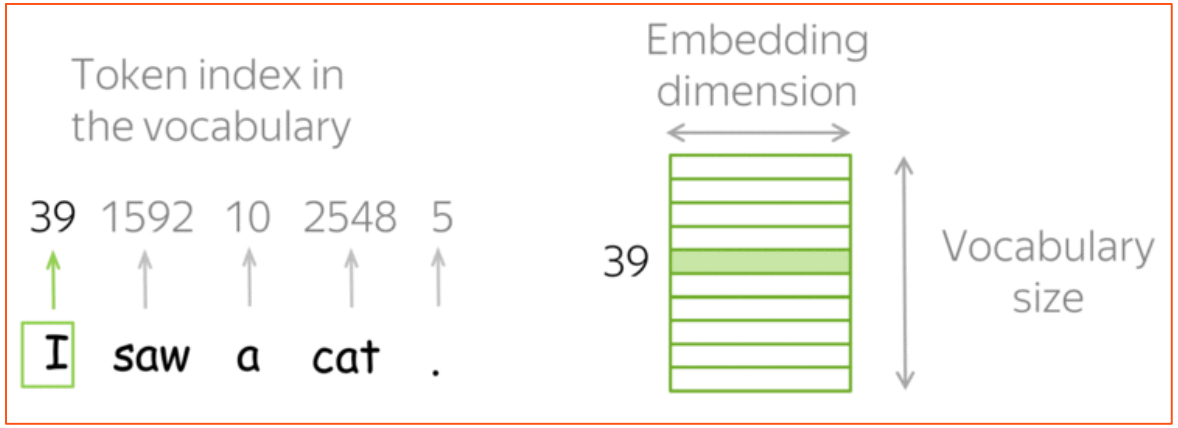
Transformer Encoder состоит из последовательных слоёв **Transformer Encoder Layers**



https://jalammar.github.io/illustrated-transformer/

Embeddings

***V* = {_, *ма, мы, ла, ра, му, боб, ры, доб, рот, ой, пол, ны,* <bos>, <eos>, <pad>}**



$E \in \mathbb{R}^{n \times d}$ – матрица эмбеддингов,

n – длина последовательности,

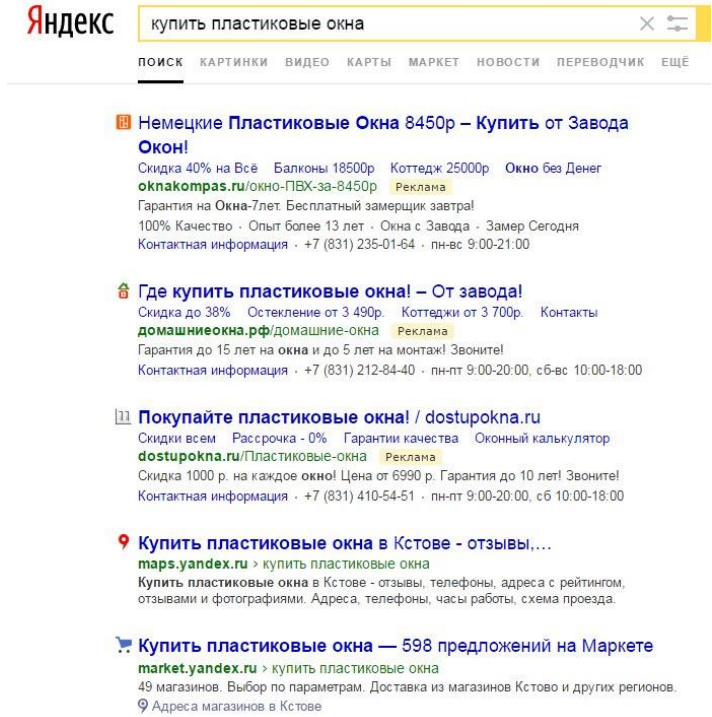
d – внутренняя размерность

Текст	Последовательность токенов	Последовательность номеров
мама мыла раму	ма,ма,_,мы,ла,_,ра,му	1,1,0,2,3,0,4,5

https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Self-Attention

Внутри *Self-Attention* поддерживаются три сущности: **Query, Key, Value**, помогающие находить связи между токенами

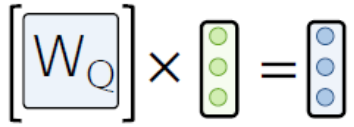


Аналогия с работой поисковой системы:

1. Вы вводите в строку поиска **Query**
2. Поисковик сравнивает ваш запрос **Query** с заголовками статей – это **Key**
3. Когда находится совпадение заголовка и запроса, поисковик выдает вам текст самой статьи – это **Value**

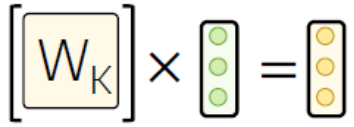
Self-Attention

Each vector receives three representations ("roles")



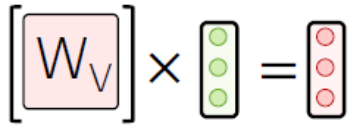
Query: vector **from** which the attention is looking

"Hey there, do you have this information?"



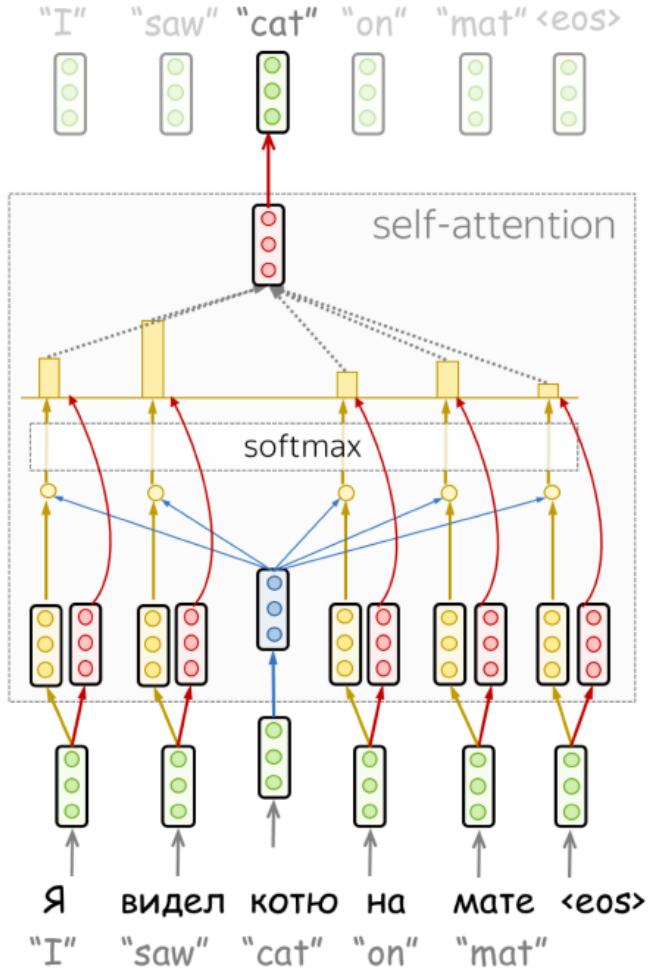
Key: vector **at** which the query looks to compute weights

"Hi, I have this information – give me a large weight!"



Value: their weighted sum is attention output

"Here's the information I have!"



https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Self-Attention

- **Query** – текущий токен, для которого мы ищем контекст

> он «спрашивает», как другие слова связаны с ним

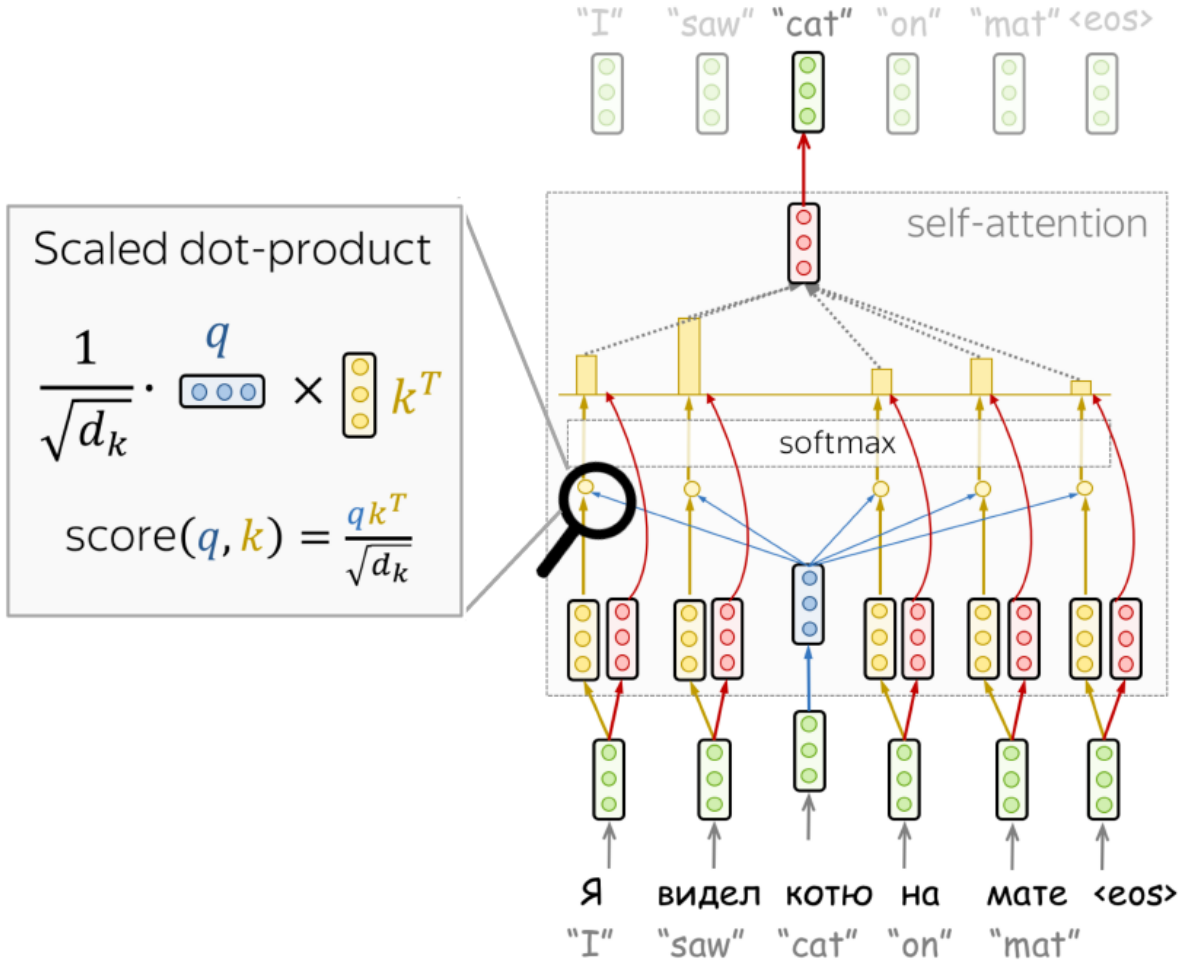
- **Key** – характеристика другого токена, с которой сравнивается **Query**

> ***Key*** служит своего рода «паспортом» токена, помогая определить, насколько он релевантен текущему запросу

- **Value** – фактическое смысловое содержание токена

> если ***Key*** подошел к ***Query***, то именно из ***Value*** извлекается важная информация, которая обогатит понимание текущего токена

Self-Attention



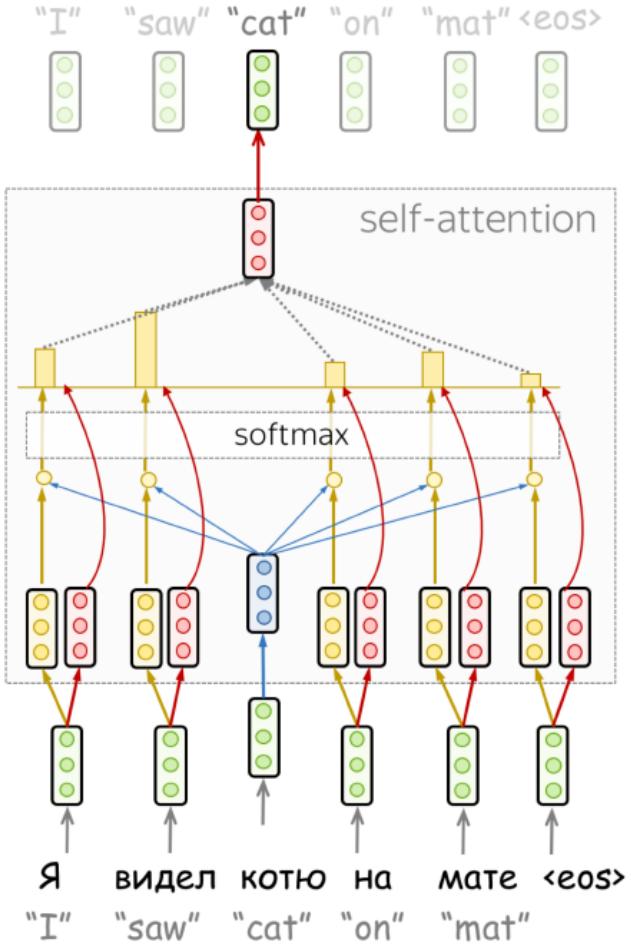
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Self-Attention

$$Attention(q, k, v) = \overbrace{softmax\left(\frac{qk^T}{\sqrt{d_k}}\right)}^{\text{Attention weights}} v$$

from (q) to (k) (v)

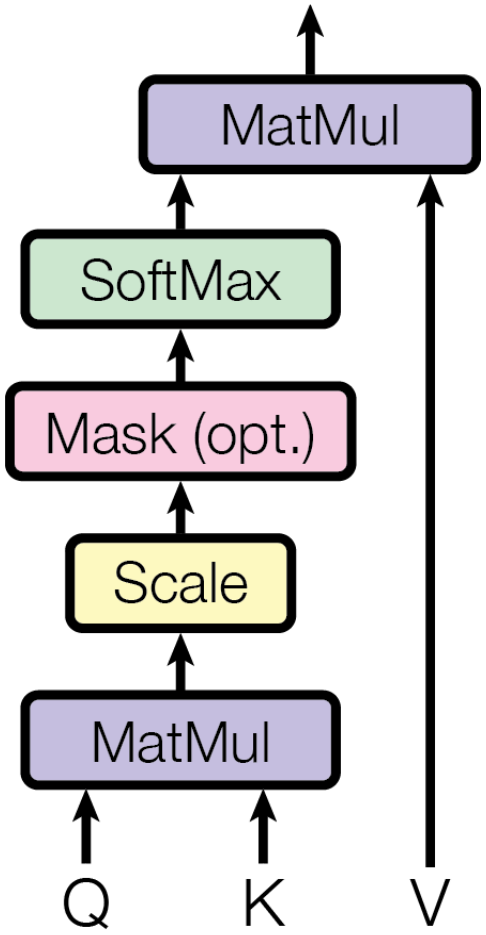
vector dimensionality of K, V



для q найти похожие k и взять соответствующие v с весами "схожести"

https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Self-Attention



$$E \in \mathbb{R}^{n \times d}, [seq_len;\ hid_dim]$$
$$W^Q, W^K, W^V \in \mathbb{R}^{d \times d}$$

$$Q = E \times W^Q$$
$$K = E \times W^K$$
$$V = E \times W^V$$

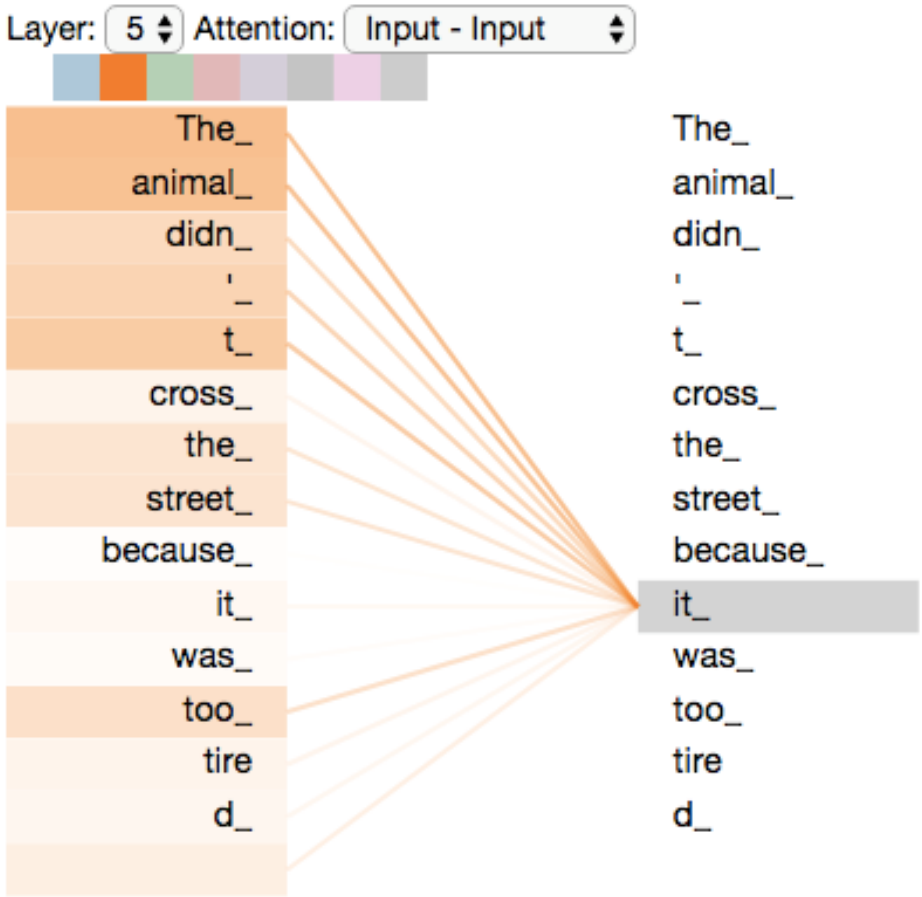
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

QK^T – attention map

$(QK^T)_{i,j}$ – "связь" i и j токенов

https://arxiv.org/pdf/1706.03762

Self-Attention



https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Self-Attention

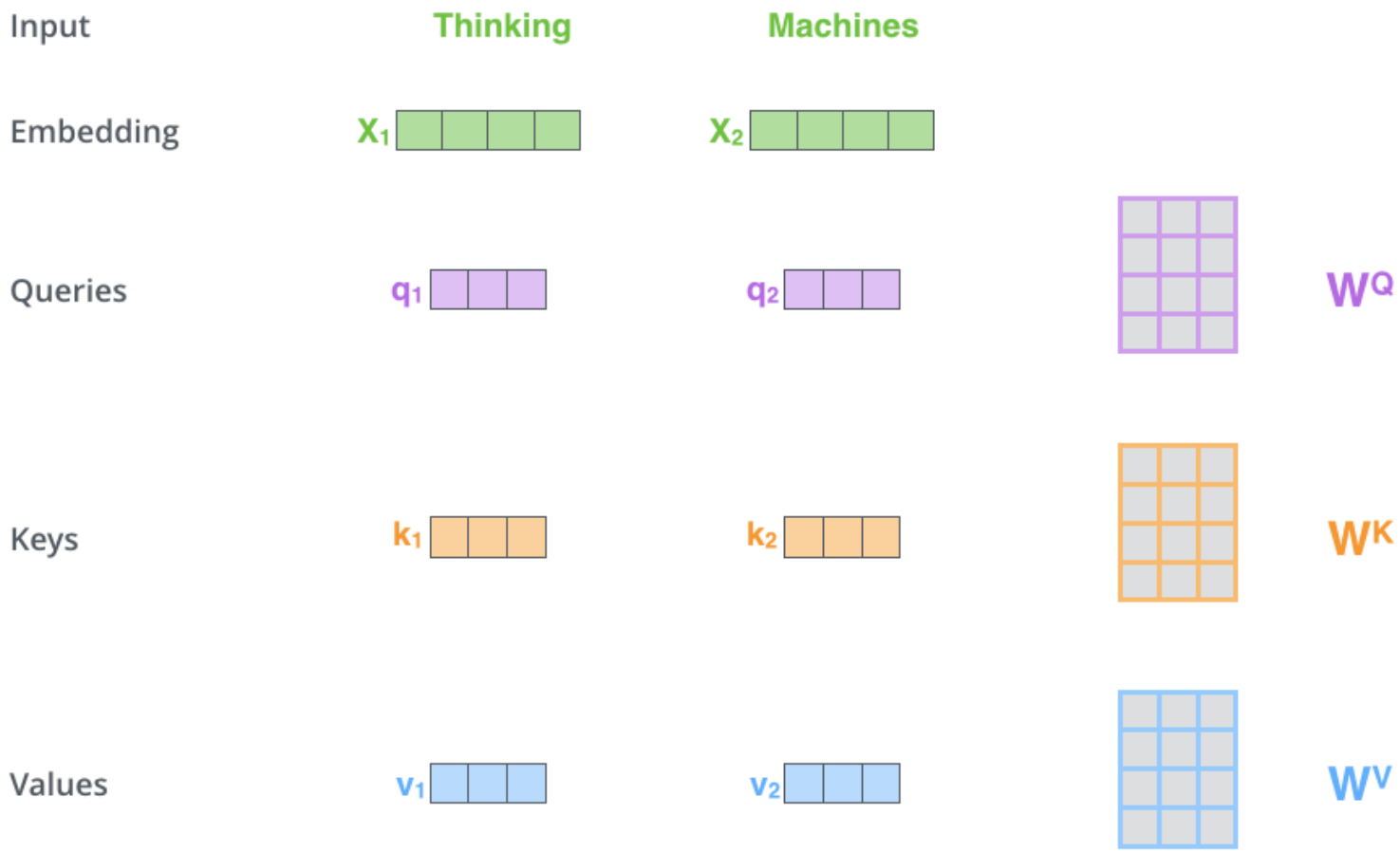
Пусть содержимое векторов q,k- это н.с.в. с распределением $\mathcal{N}(0,1)$. Тогда:

$$\mathbb{E}(< q,k >) = \mathbb{E}(\sum_{j=1}^{d} q_j * k_j) = \sum_{j=1}^{d} \mathbb{E}q_j * \mathbb{E}k_j = 0$$

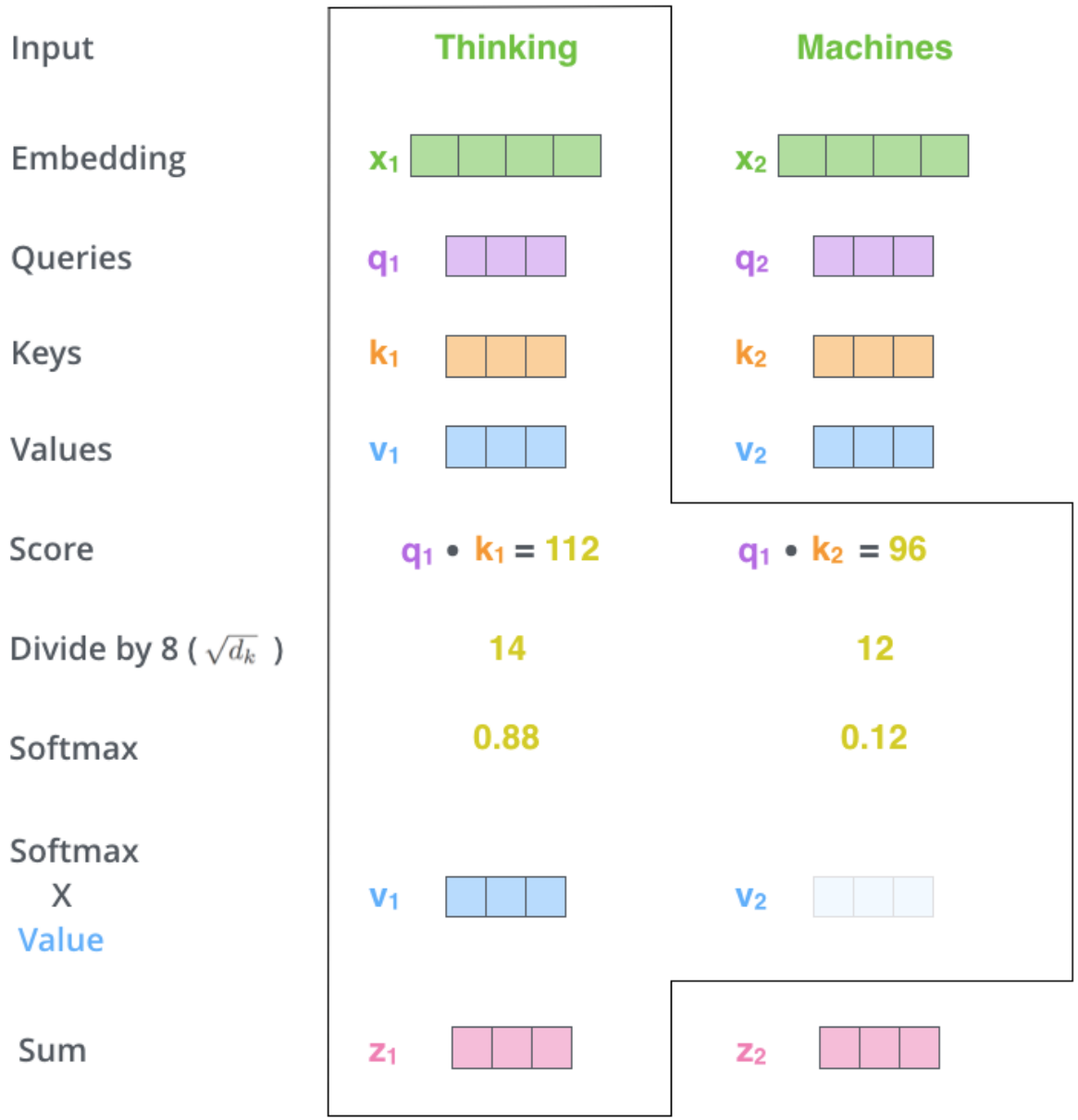
$$Var(< q,k >) = Var(\sum_{j=1}^{d} q_j * k_j) = \sum_{j=1}^{d} Var(q_j) * Var(k_j) = d$$

$$softmax(K \times Q^T) \rightarrow softmax(\frac{K \times Q^T}{\sqrt{d}})$$

Self-Attention

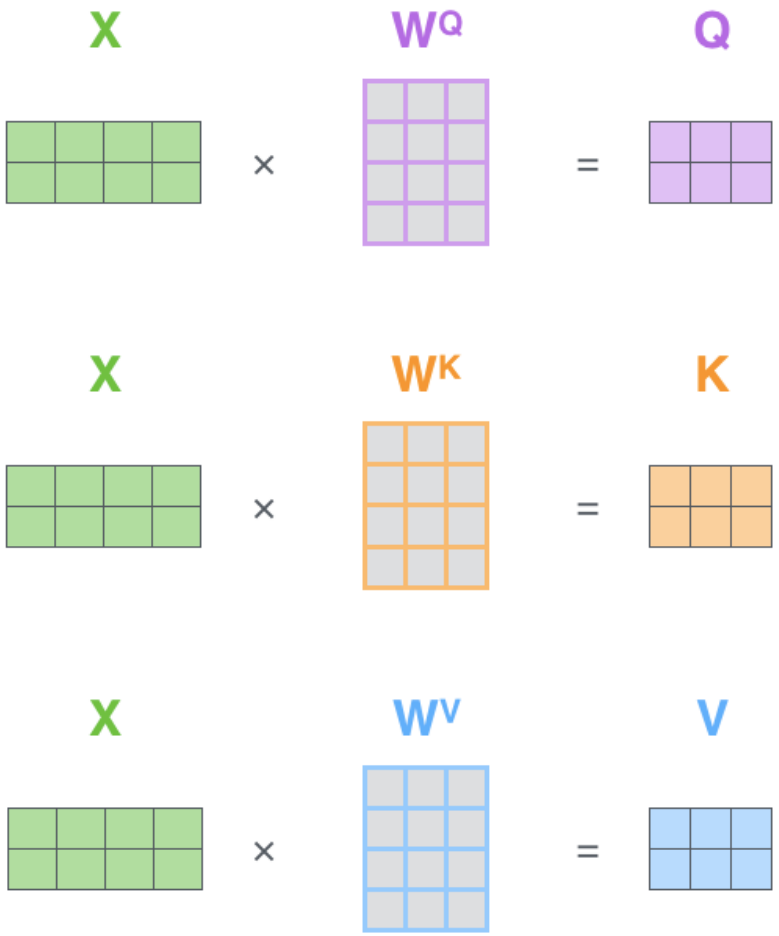


https://jalammar.github.io/illustrated-transformer/



https://jalammar.github.io/illustrated-transformer/

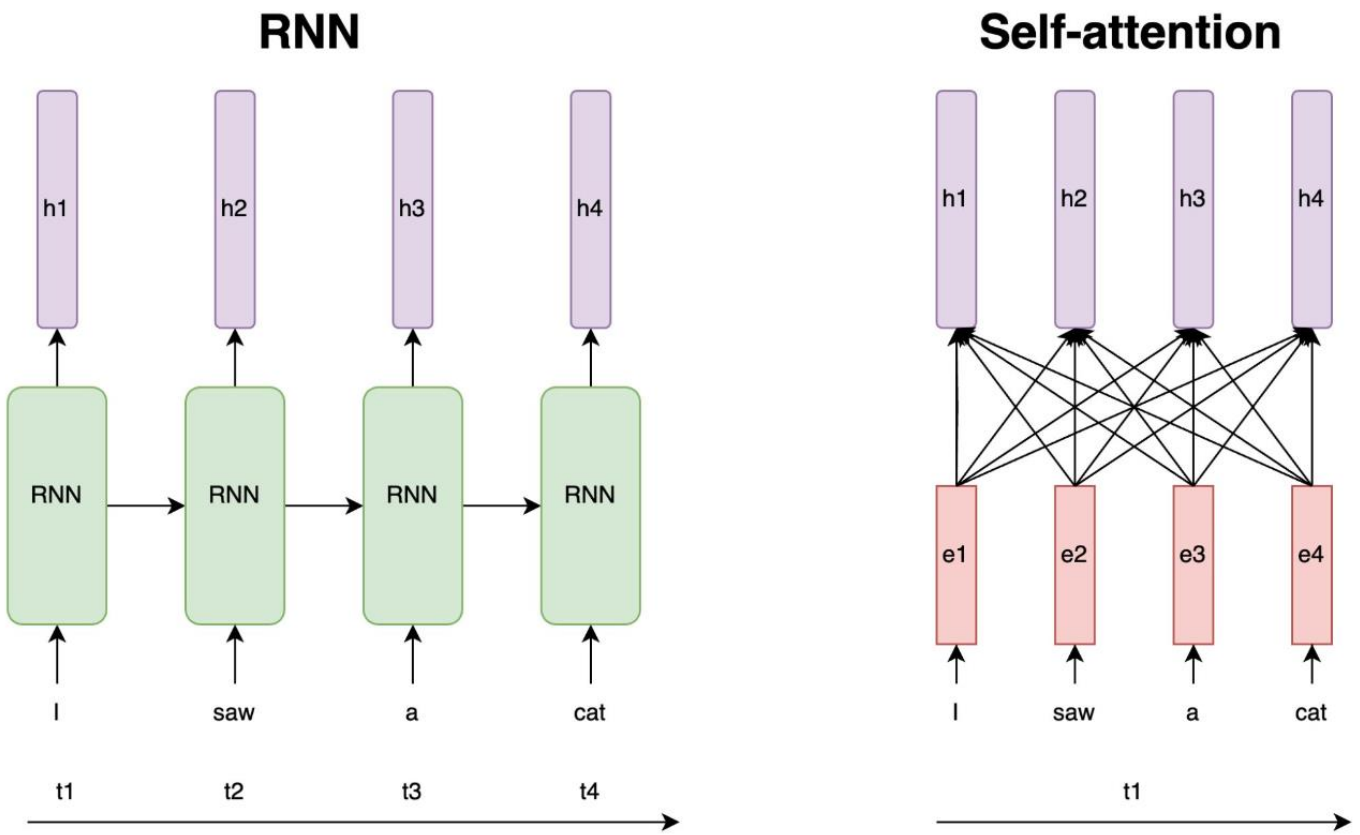
Self-Attention



$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V = Z$$

https://jalammar.github.io/illustrated-transformer/

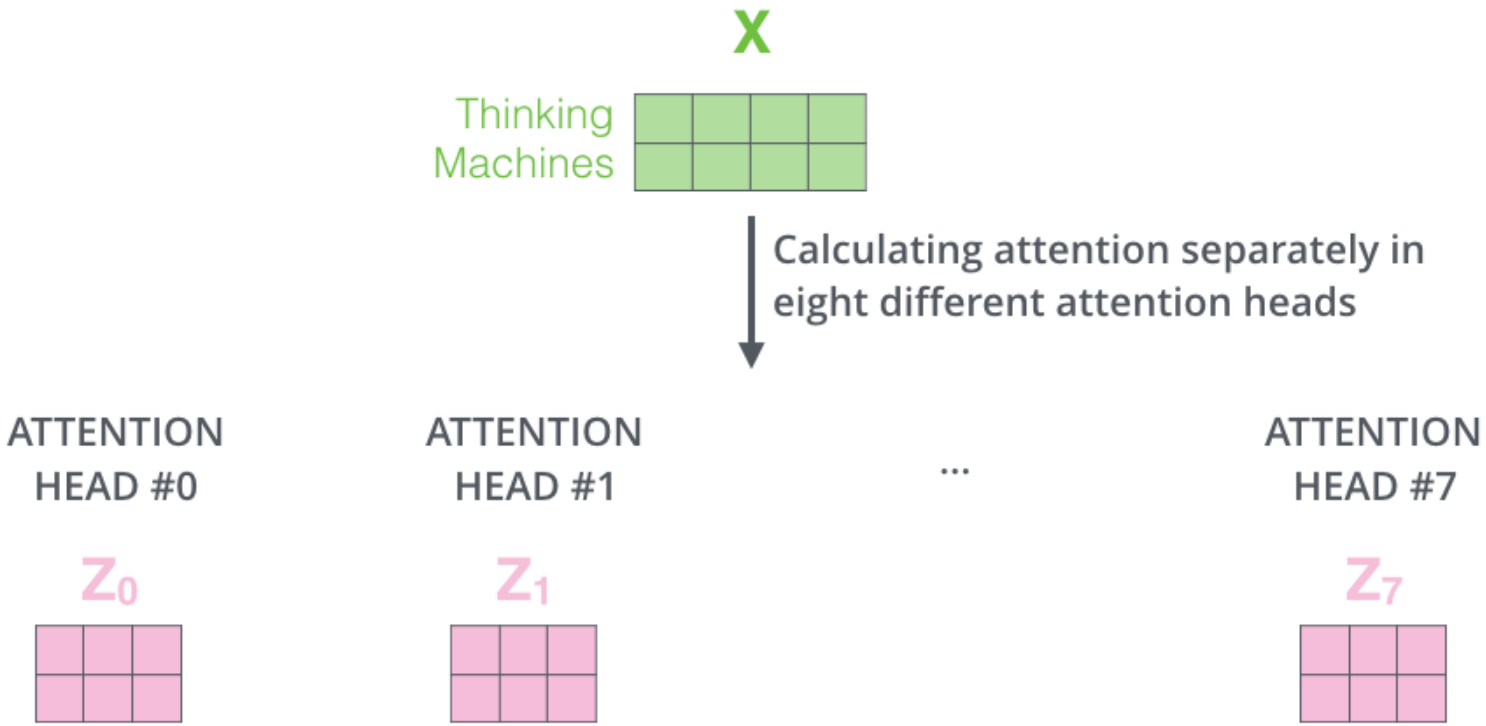
Self-Attention



https://github.com/Mashka8312/llm-course-2025-spring/blob/main/Lectures/02.pdf

Multi-Head Attention

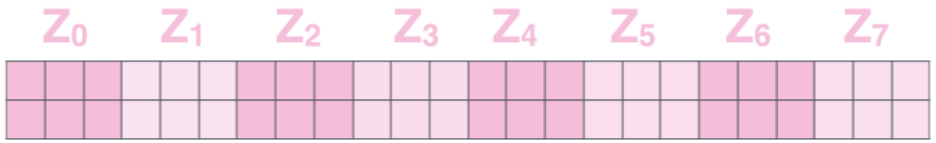
Несколько голов обеспечивают разное внимание



https://jalammar.github.io/illustrated-transformer/

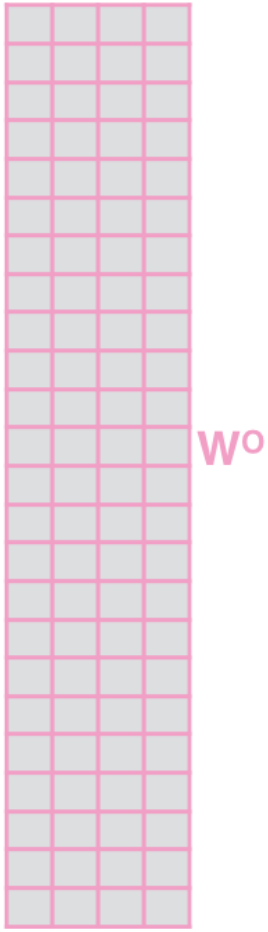
Multi-Head Attention

1) Concatenate all the attention heads



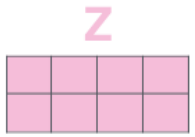
2) Multiply with a weight matrix W^O that was trained jointly with the model

X



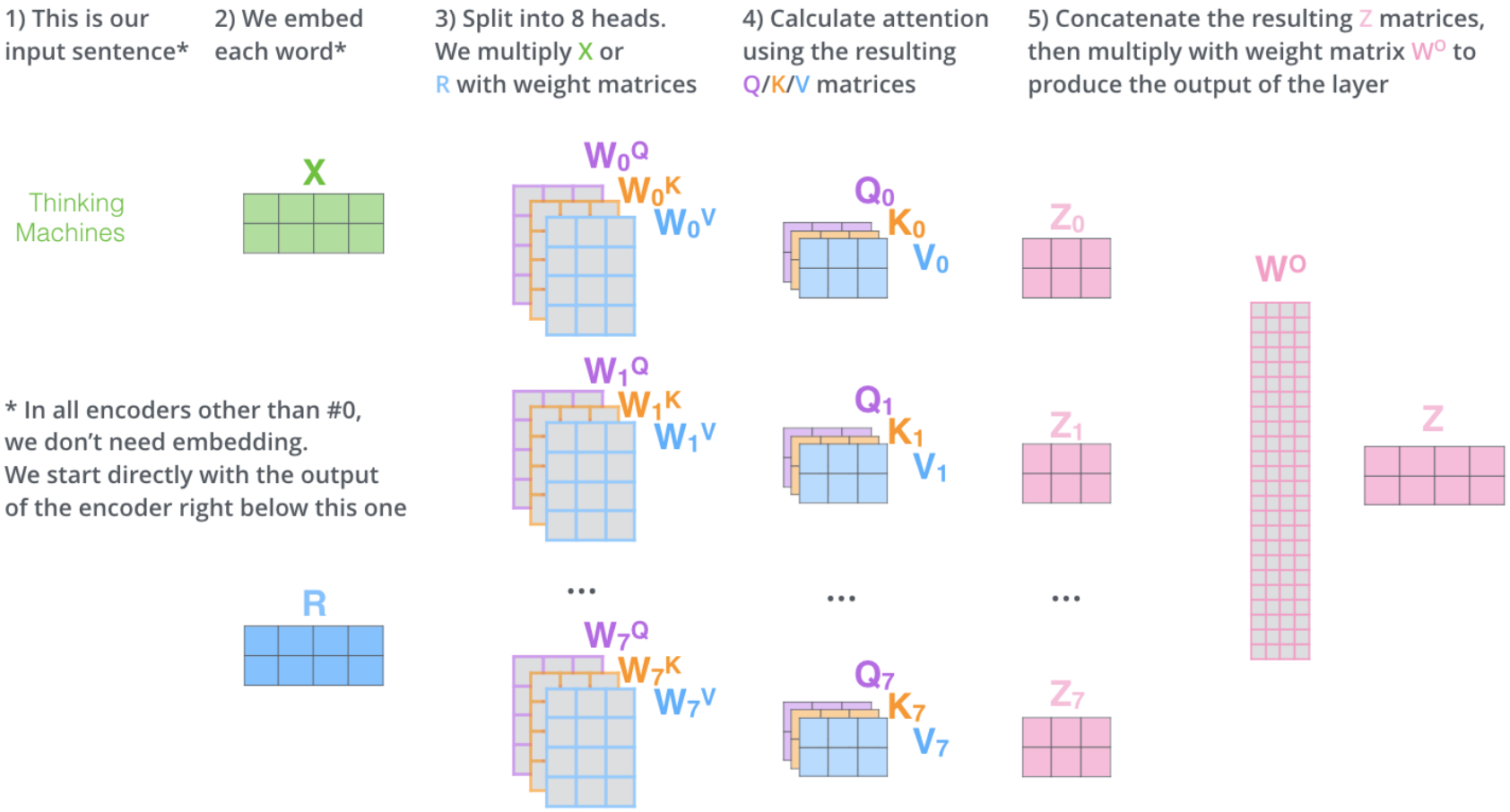
3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

=



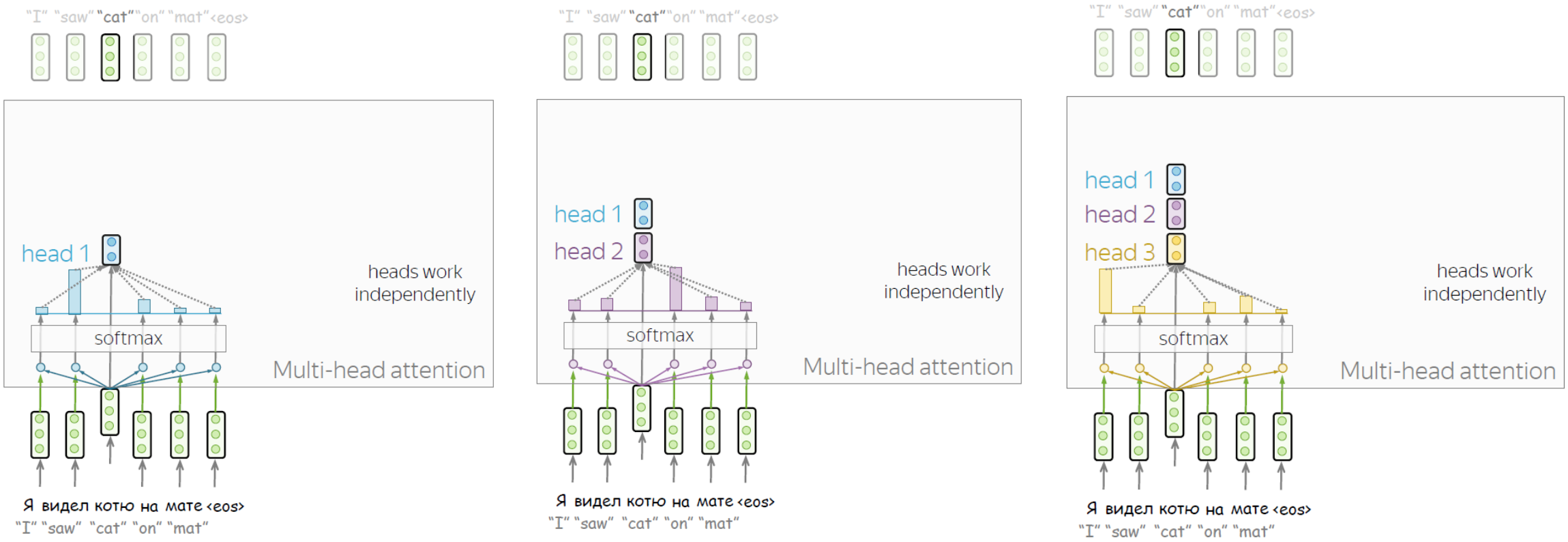
https://jalammar.github.io/illustrated-transformer/

Multi-Head Attention



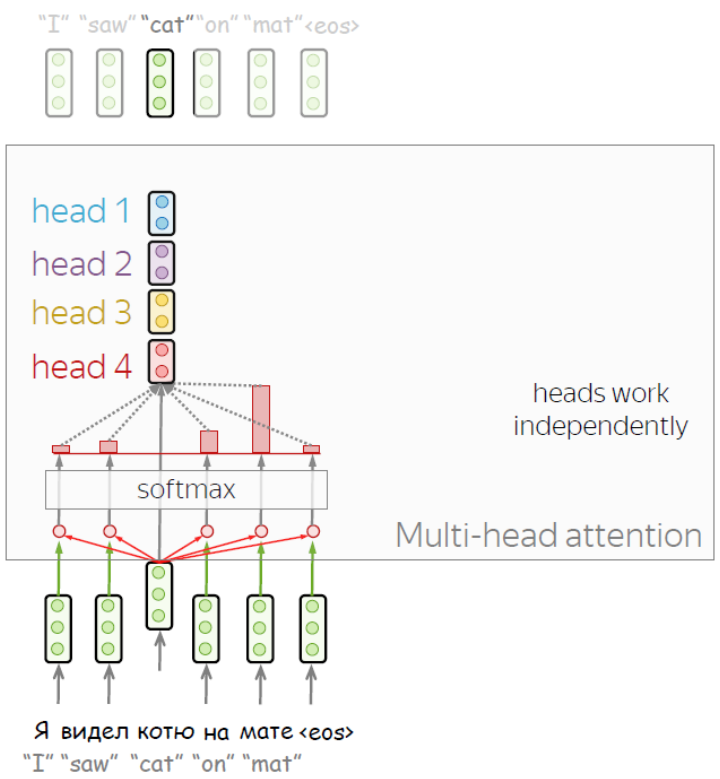
https://jalammar.github.io/illustrated-transformer/

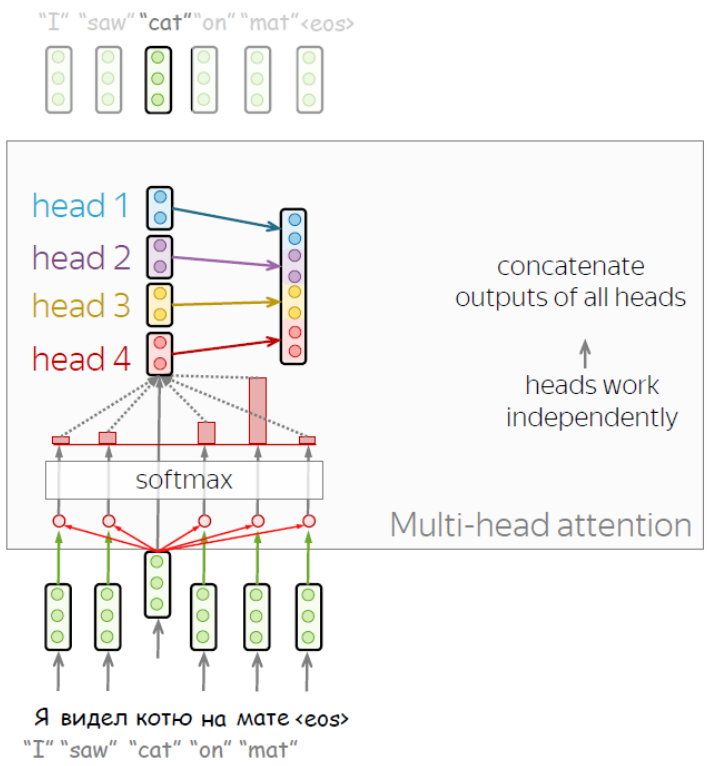
Multi-Head Attention

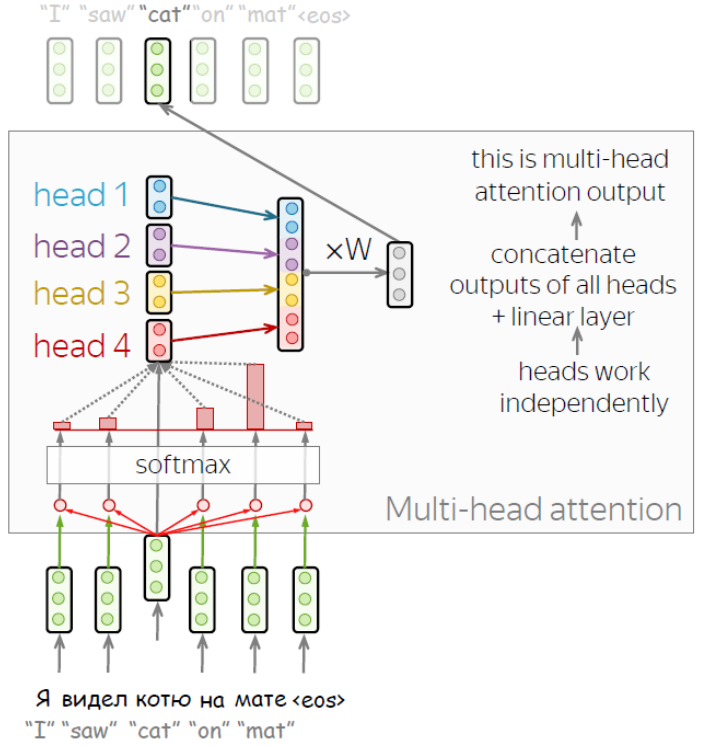


https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Multi-Head Attention

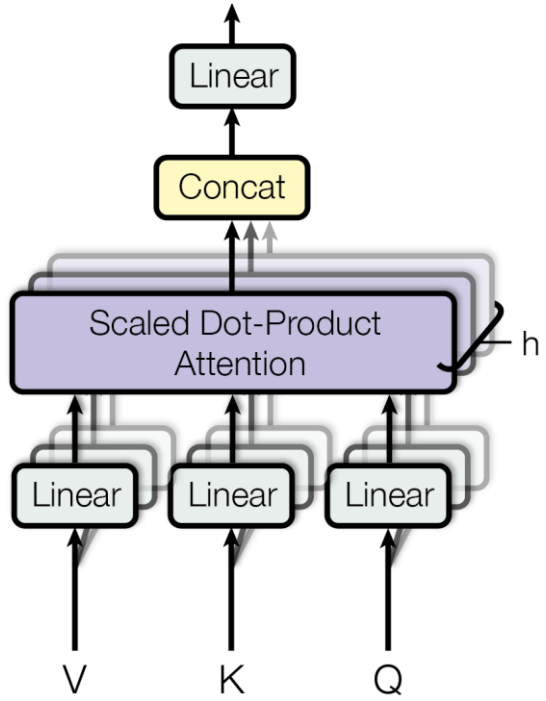






https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Multi-Head Attention



Было: $Q, K, V \in \mathbb{R}^{\mathbf{d} \times \mathbf{d}}$

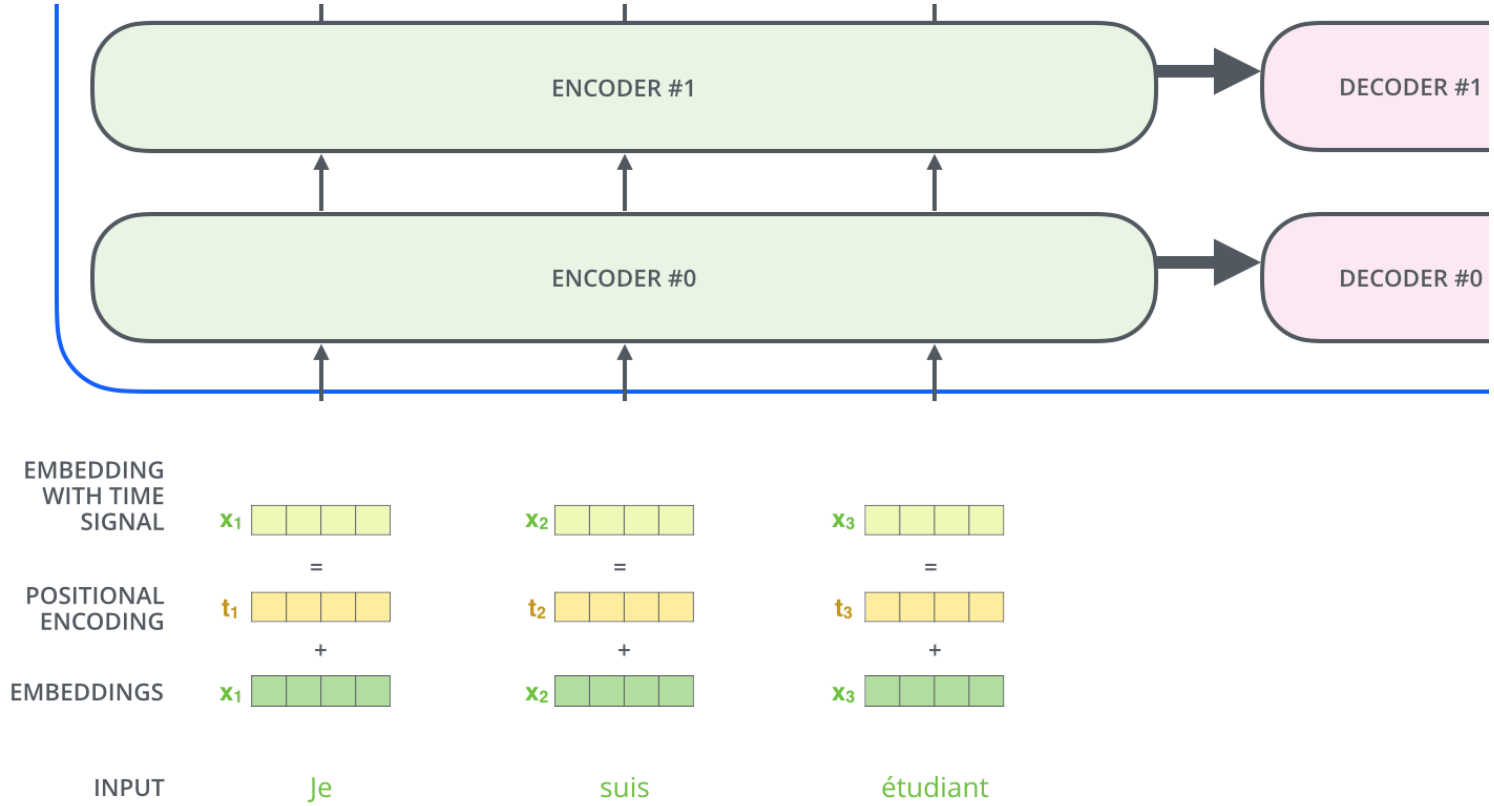
Стало: k матриц $(Q_1, K_1, V_1), \ldots, (Q_k, K_k, V_k)$. Каждая матрица $\in \mathbb{R}^{\mathbf{d} \times \frac{\mathbf{d}}{\mathbf{k}}}$

$$MultiHeadAttention = [SelfAttention_1(x), SelfAttention_2(x), \ldots, SelfAttention_k(x)] * O, O \in \mathbb{R}^{\mathbf{d} \times \mathbf{d}}$$

https://github.com/Mashka8312/llm-course-2025-spring/blob/main/Lectures/02.pdf

Positional Encoding

Не хватает позиционной информации

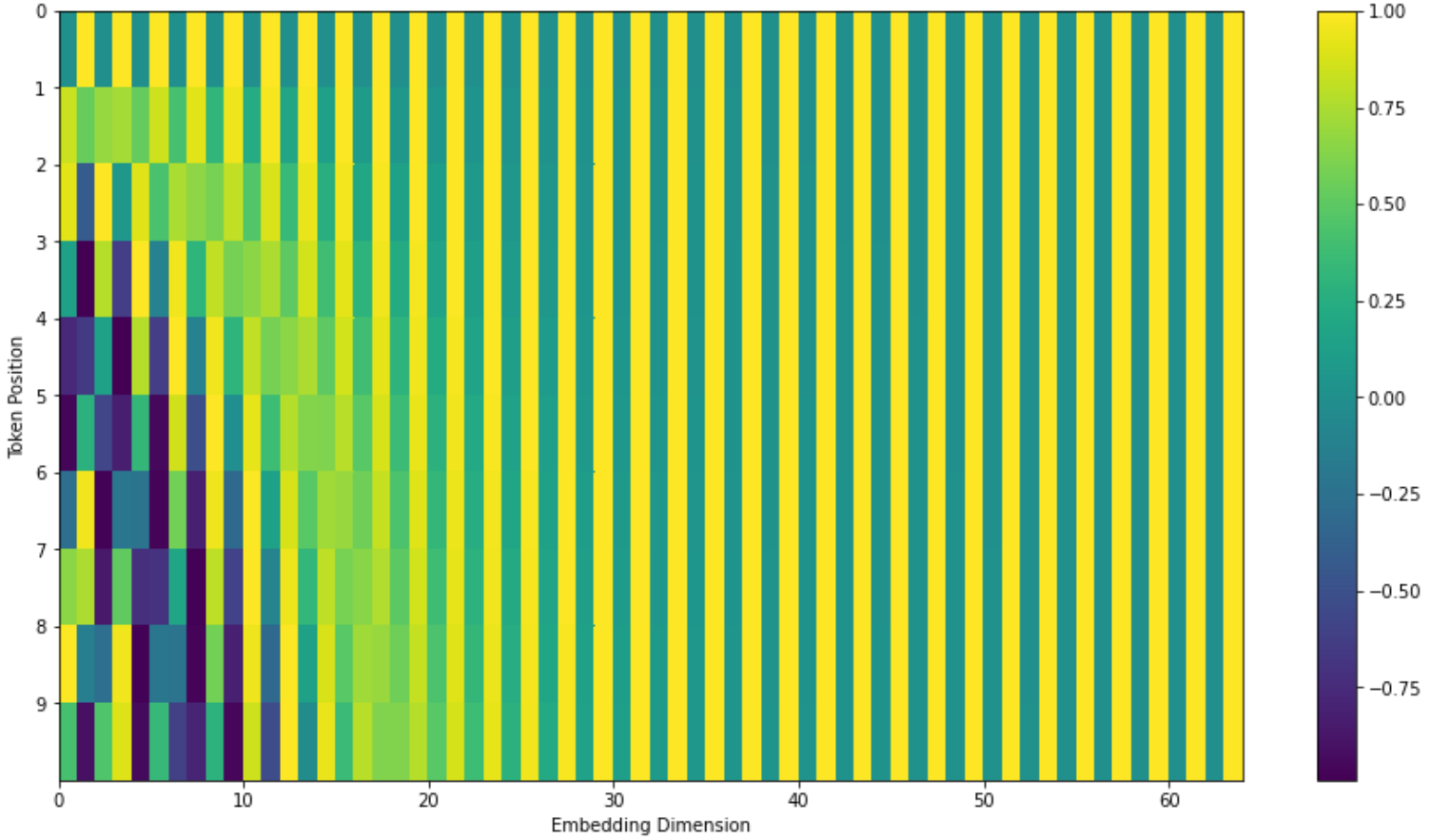


https://jalammar.github.io/illustrated-transformer/

Positional Encoding

- $\tilde{E} = E + PE$

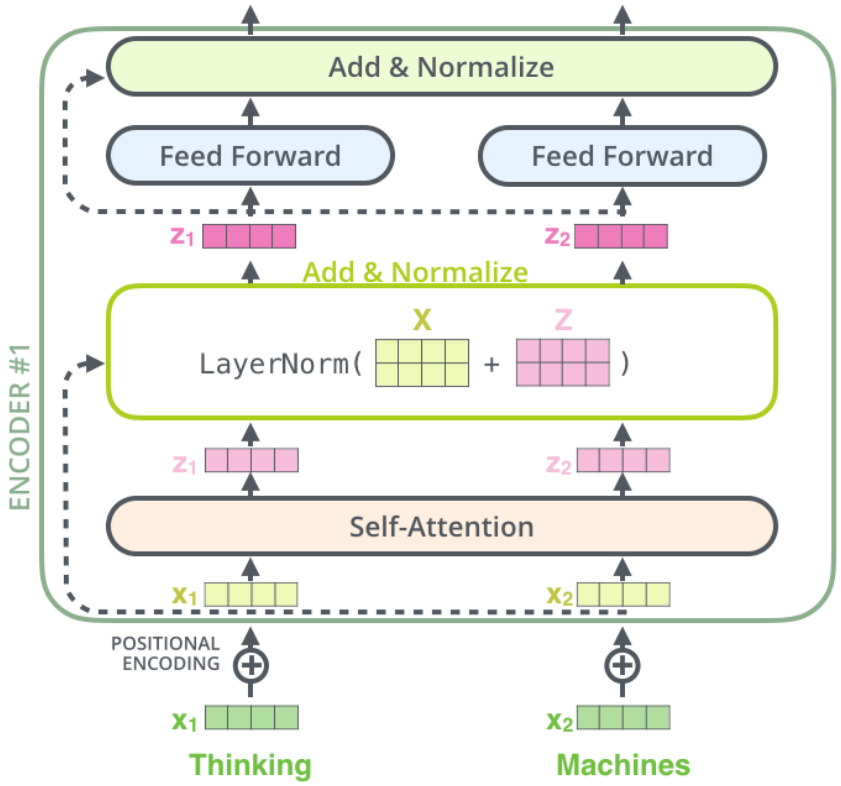
$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

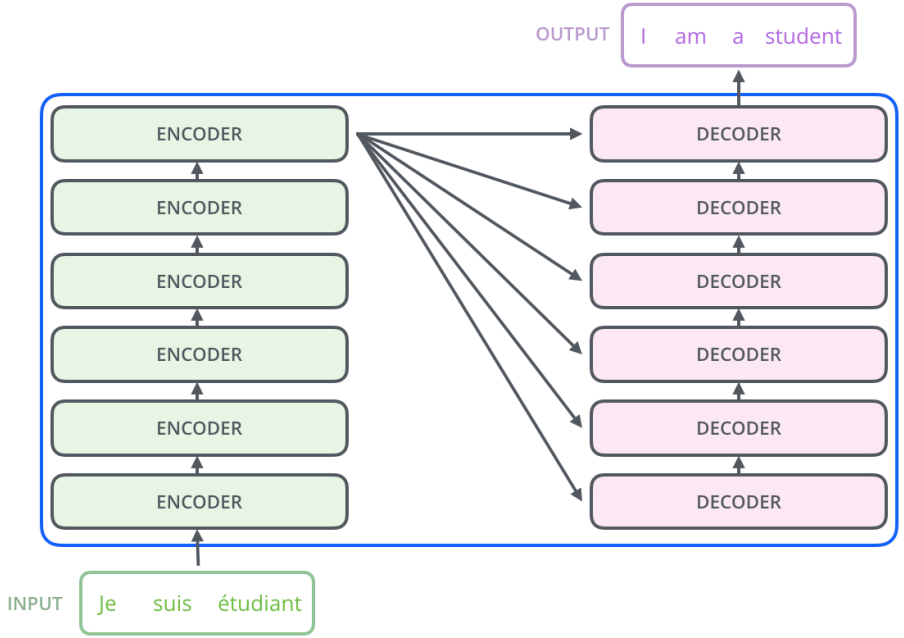


https://arxiv.org/pdf/1706.03762

Transformer. Encoder

Transformer Encoder состоит из последовательных слоёв **Transformer Encoder Layers**

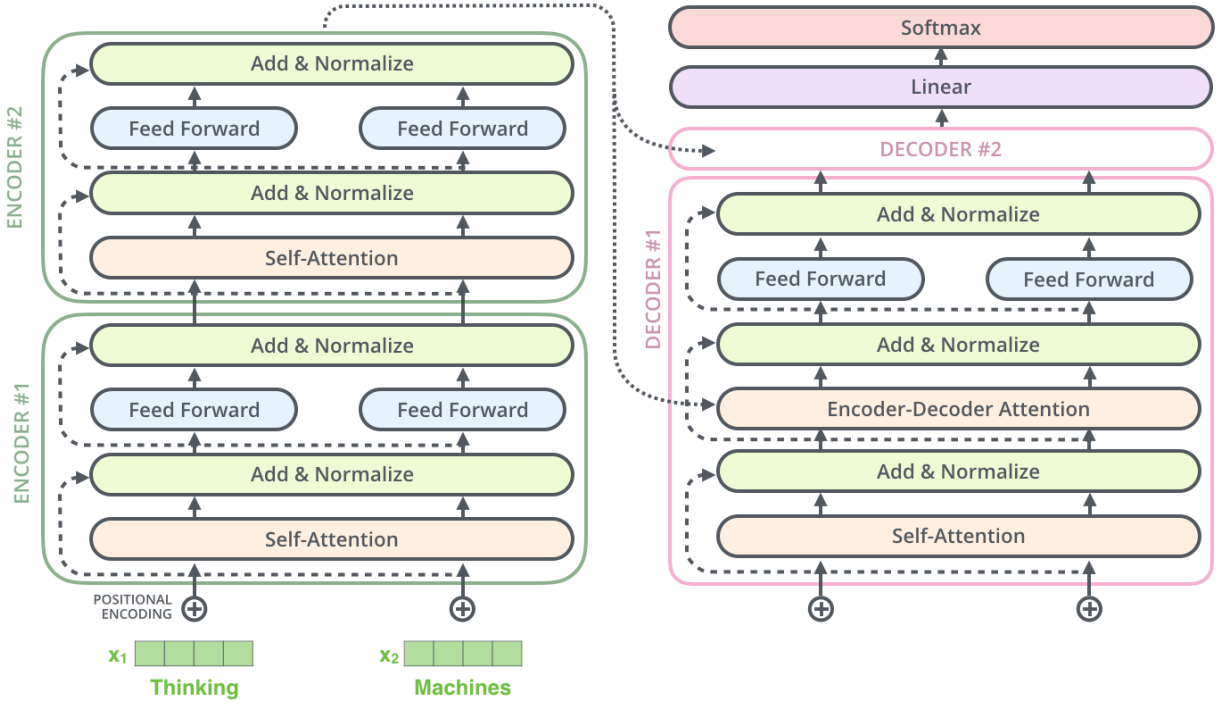


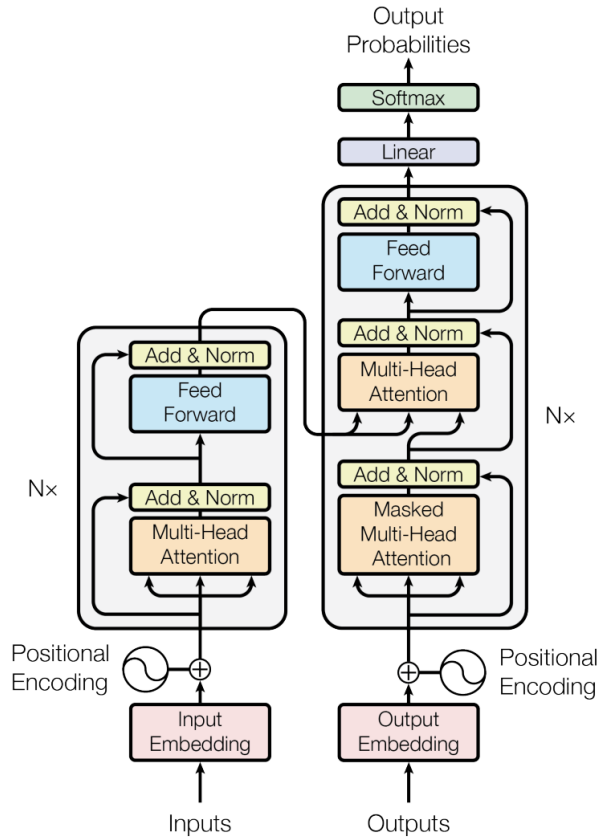


https://jalammar.github.io/illustrated-transformer/

Transformer. Decoder

Transformer Decoder состоит из последовательных слоёв **Transformer Decoder Layers**

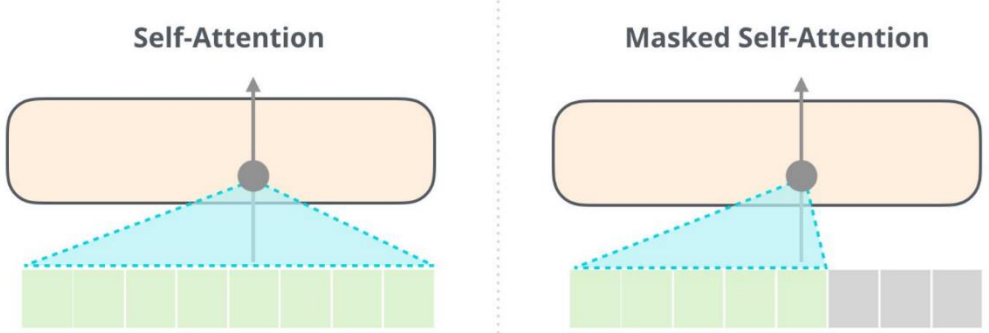


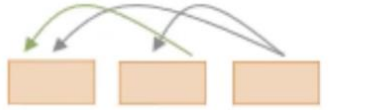


https://jalammar.github.io/illustrated-transformer/

Masked Self-Attention

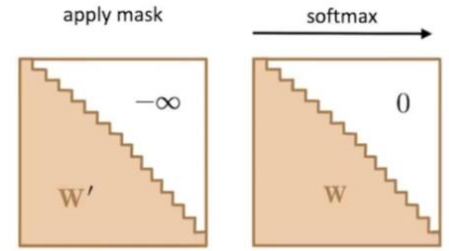
Отличие работы self-attention в энкодере и в декодере:





Masked Decoder Self-Attention

We need mask attention matrix to not look at future tokens during training:



$W'_{ij} \leftarrow -\infty \text{ if } j > i$ $W = \text{softmax}(W')$

Attend to the previous words in the generated output sequence

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + Mask\right)V$$

https://github.com/Mashka8312/llm-course-2025-spring/blob/main/Lectures/02.pdf

Masked Self-Attention

Предположим, у нас есть фраза из 4 токенов: «**кот спит на диване**»

Матрица маски:

$$Mask = \begin{pmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Строки отвечают за текущий токен (запрос **Q**)

Столбцы отвечают за все токены (ключи **K**), на которые **Q** смотрит

Masked Self-Attention

Предположим, у нас есть фраза из 4 токенов: «**кот спит на диване**»

$$Mask = \begin{pmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

- 1 строка («**кот**»): Видит только себя. Остальные токены для него закрыты
- 2 строка («**спит**»): Видит слово «**кот**» и себя. Токены «**на**» и «**диване**» скрыты
- 3 строка («**на**»): Видит «**кот**», «**спит**» и себя. Закрыт только токен «**диване**»
- 4 строка («**диване**»): Видит всю фразу целиком, так как это последний токен

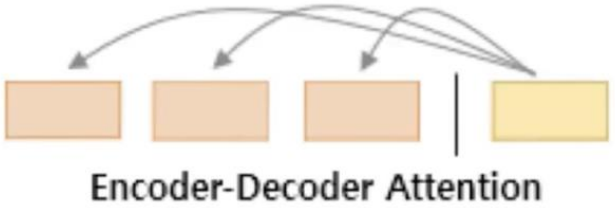
Masked Self-Attention

После сложения и *Softmax*:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + Mask\right)V = Weights \times V =$$

$$\begin{pmatrix} 1.00 & 0.00 & 0.00 & 0.00 \\ 0.50 & 0.50 & 0.00 & 0.00 \\ 0.33 & 0.33 & 0.33 & 0.00 \\ 0.25 & 0.25 & 0.25 & 0.25 \end{pmatrix} \times V$$

Encoder-Decoder Attention

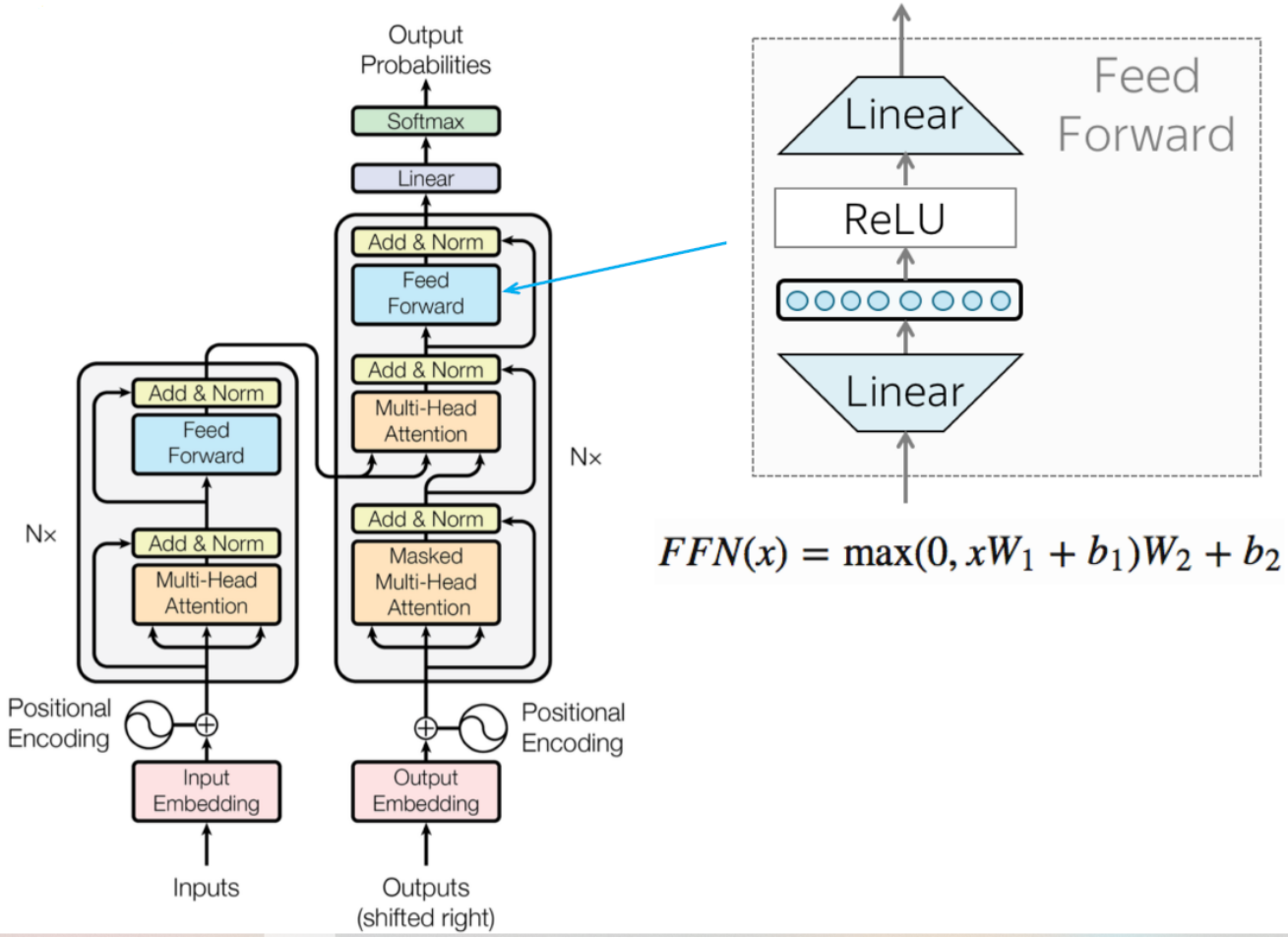


This is not self-attention:

- Q from decoder
- K, V from encoder

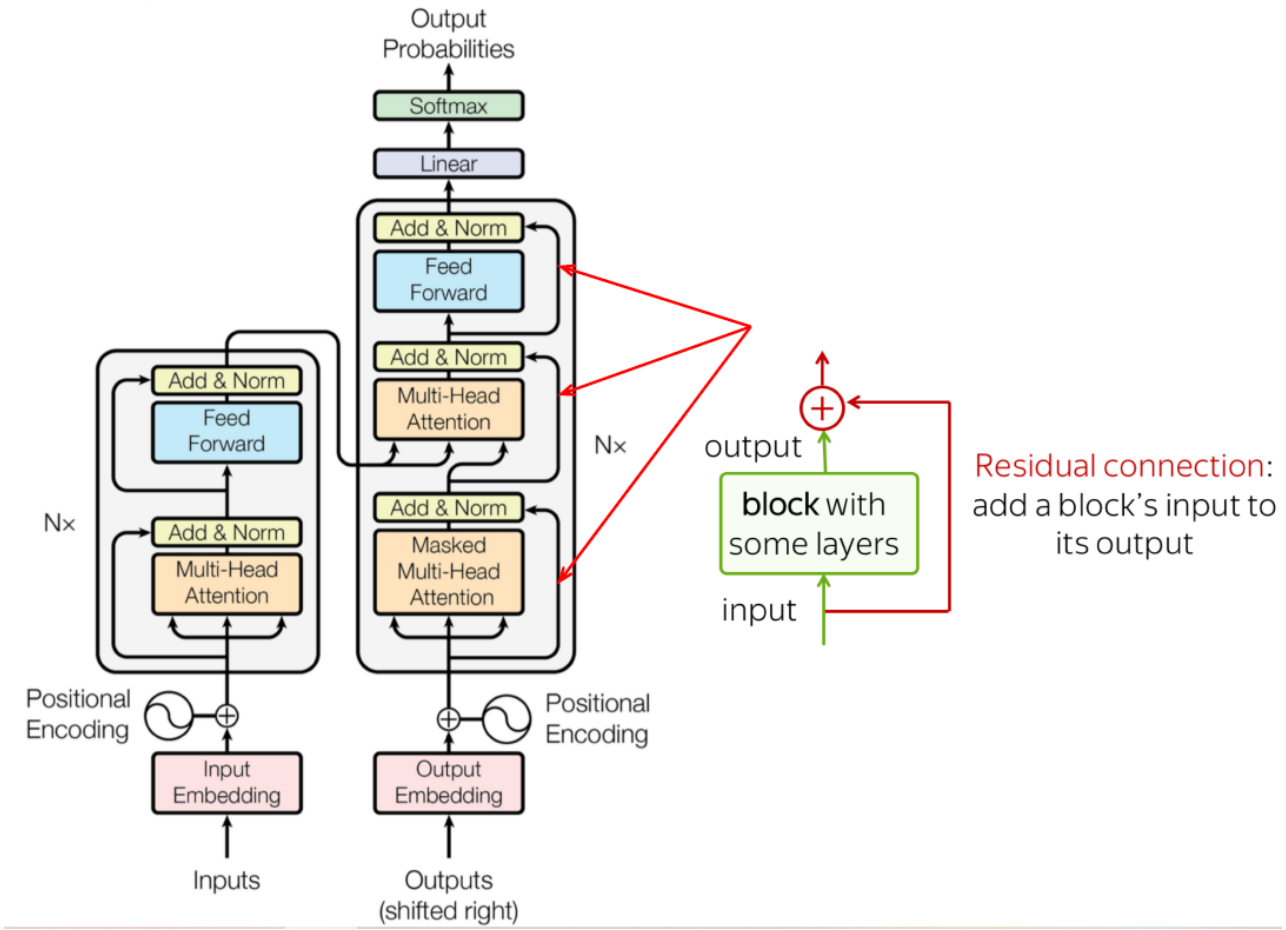
Attend to tokens from the input sequence relevant for the generation of the next output token

Feed Forward Block



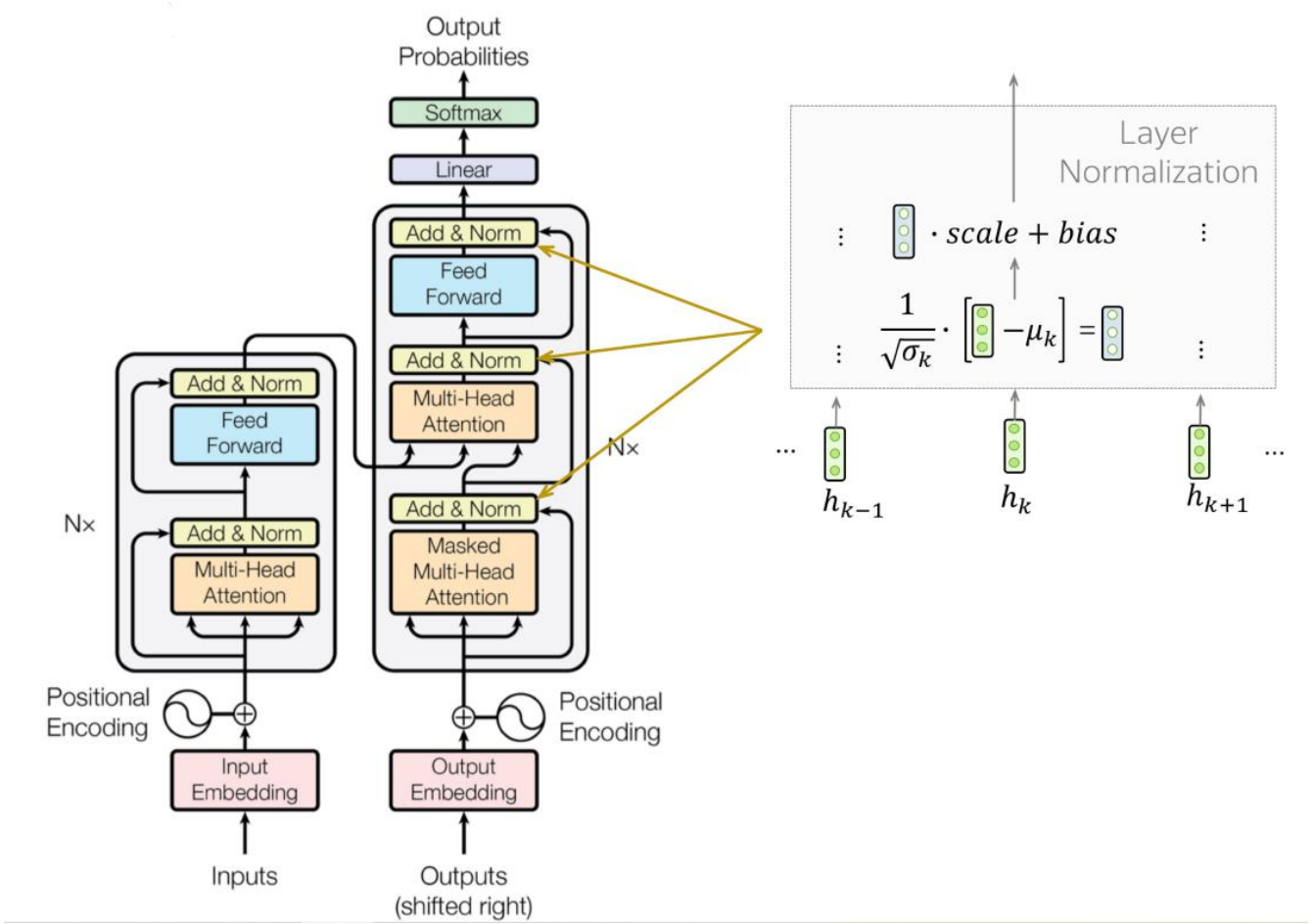
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Residual Connections



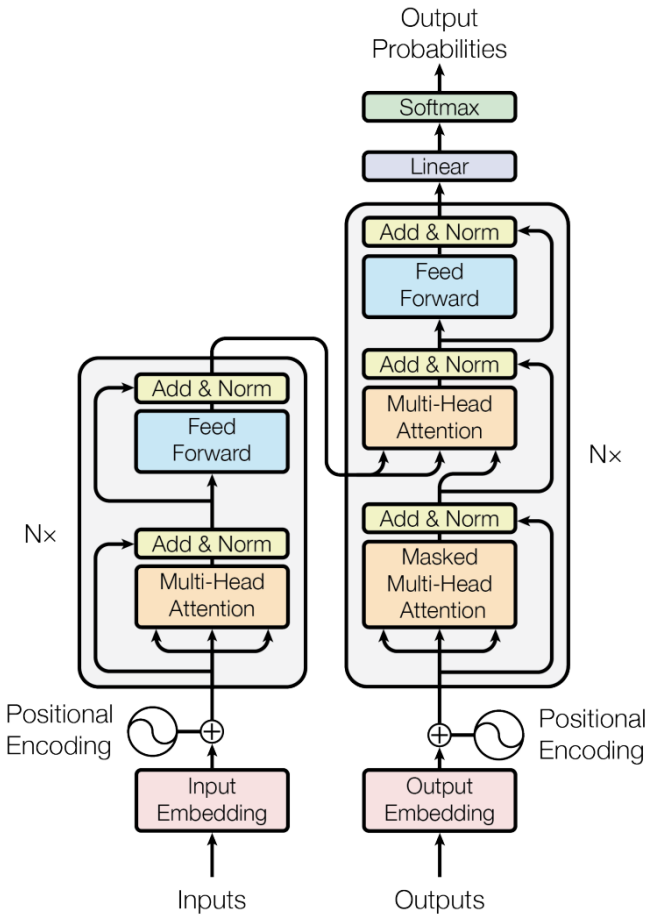
https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Layer Normalization



https://github.com/yandexdataschool/nlp_course/tree/2021/week04_seq2seq

Transformer

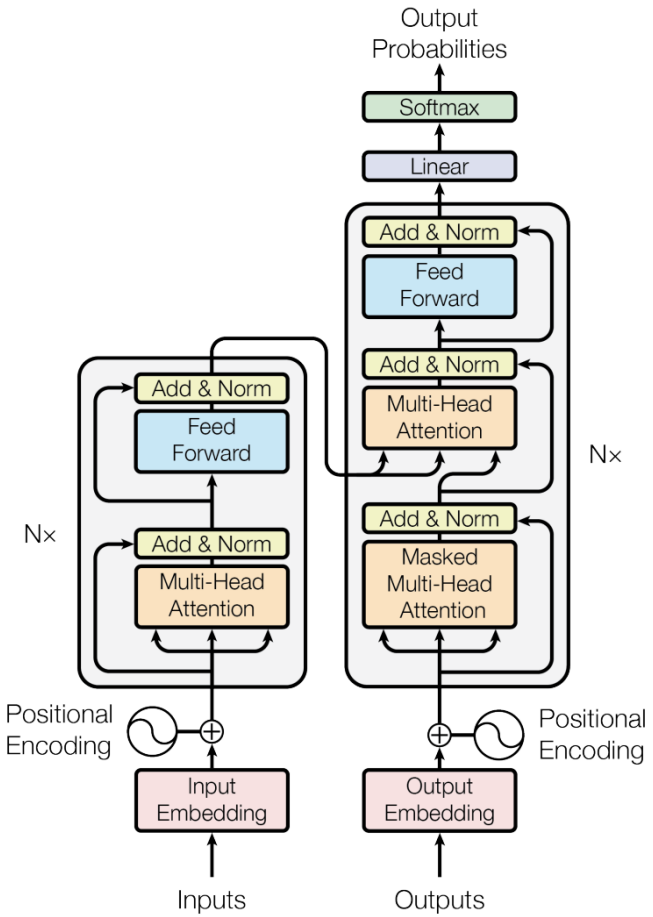


https://huggingface.co/learn/llm-course/chapter1/5

BERT, GPT

BERT

Encoder

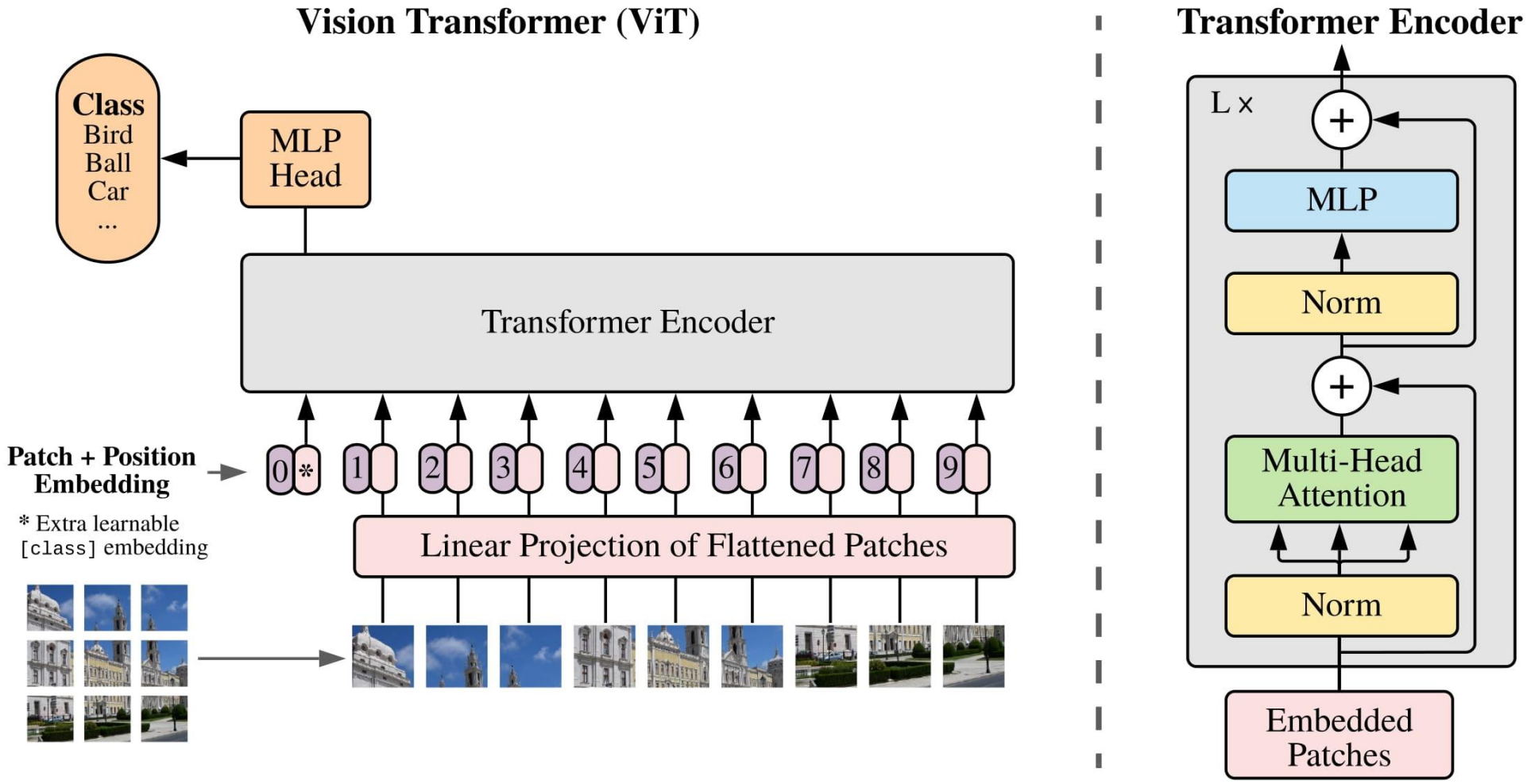


GPT

Decoder

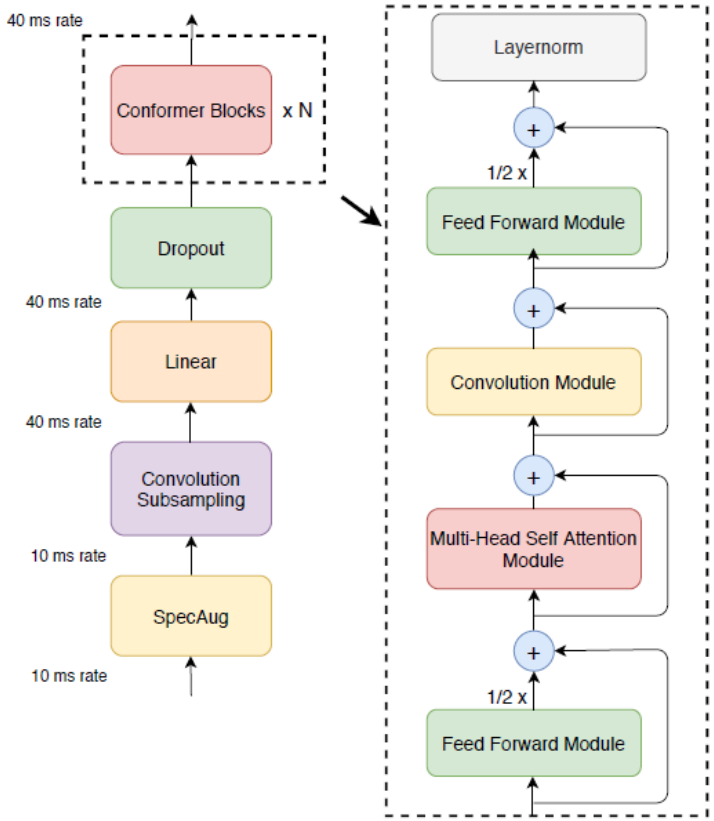
https://huggingface.co/learn/llm-course/chapter1/5

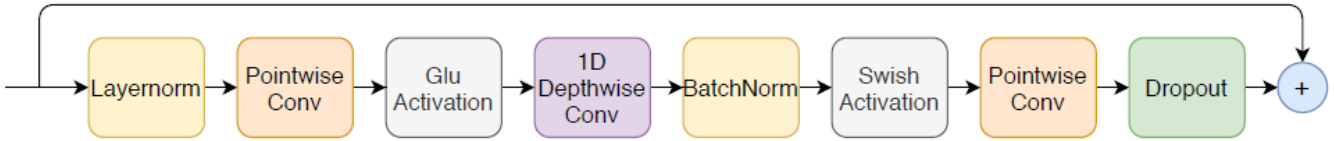
Image Classification: Vision Transformer (ViT)

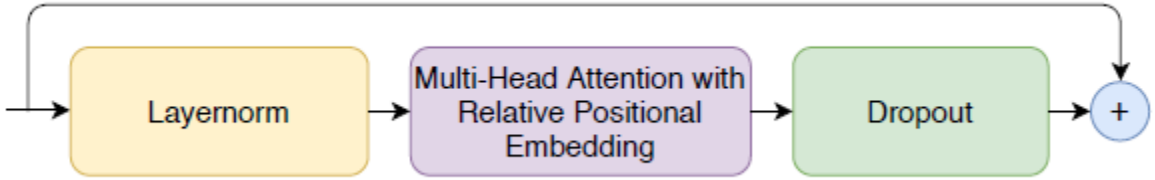


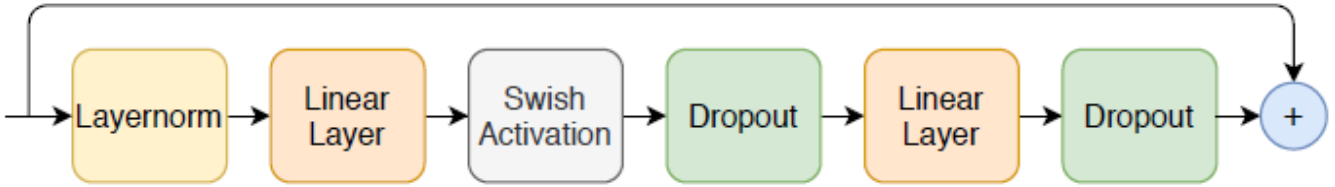
https://huggingface.co/learn/llm-course/chapter1/5

Automatic Speech Recognition: Conformer











Спасибо за внимание!

@evgeniidarkhanov